Models Rmd

2026-06-26

Models

- Package Installation

```
library("rethinking")
library(dplyr)
library(dagitty)
```

Causal effects of the DAG

```
kdag<-dagitty("dag {
  S-> K <- T
  T -> S
  T -> SL -> K
}")
adjustmentSets(kdag, exposure="SL", outcome = "K")# This shows the backdoor path and how to stratify
```

```
## { T }
```

Synthetic Data

- create a synthetic data with assumptions. See overview.pdf to check the data relationship.

```
# Function of SL by Y
# This is arbitrary made by me, immitating the research by Susumu Tanabe and Rei Nakashima in 2026
# Y is between 0 and 1, which represents 16,000 years ago to 2,400 years ago.
myFunc <- function(x) {
case_when(
    0 <= x & x <= (-9+ 16)/(13.6)                          ~ -(x * (-13.6)) * 60 / 7 - 80,
    (-9.000+ 16)/(13.6) < x  & x <= (-7.500 + 16)/(13.6)   ~ -(x * (-13.6) + 7) * 46 / 3 - 20,
    (-7.500+ 16)/(13.6) < x   & x <= (-4.3+ 16)/(13.6)       ~ 3,
    (-4.3 + 16)/(13.6) < x   & x <= (-3.000+ 16)/(13.6)    ~ -(x * (-13.6) + 11.7) * (-60) / 13 + 3,
    (-3.000+ 16)/(13.6) < x   & x <= (-2.400+ 16)/(13.6)    ~ -(x * (-13.6) + 13) * 5 / 3 - 1,
    TRUE                                                    ~ 1
  )
}
curve(myFunc(x), col = 2, main = "Relative Sea Level and Year")

# Initialize Y
Y<- runif(200, 0, 1)

# T and SL caused by Y
T<- NULL
SL<-NULL
for (i in 1:length(Y)) {
```

```
    T[i]<-rpois(1,  Y[i]* 25)
    if(T[i] < 1) {T[i]<- 1}
    else if(T[i] > 25) {T[i]<- 25} # Make sure T is between 1 and 25
    SL[i]<- rnorm(1, myFunc(Y[i]), 5)
}

SL <- (SL - min(SL)) / (max(SL) - min(SL)) # to estimate easily, make it between 0 and 1

# Check the data
plot(T, SL, main = "Synthetic Data Plot(T and SL)")
plot(Y, T, main = "Synthetic Data Plot(Y and T)")
plot(Y, SL, main = "Synthetic Data Plot(Y and SL)")

# initialize dS with random numbers
dS<-rbeta(length(T), 1.2,1) * 200
# S is affected by Y and dS (omit T as T and S are correlated by Y)
S<-NULL
for(i in 1:length(T))S[i] <- rnorm(1, 5*Y[i] -dS[i]/50, 0.1)/25 + 0.3

# Check the data
range(S) # try to make less than 0.5
```

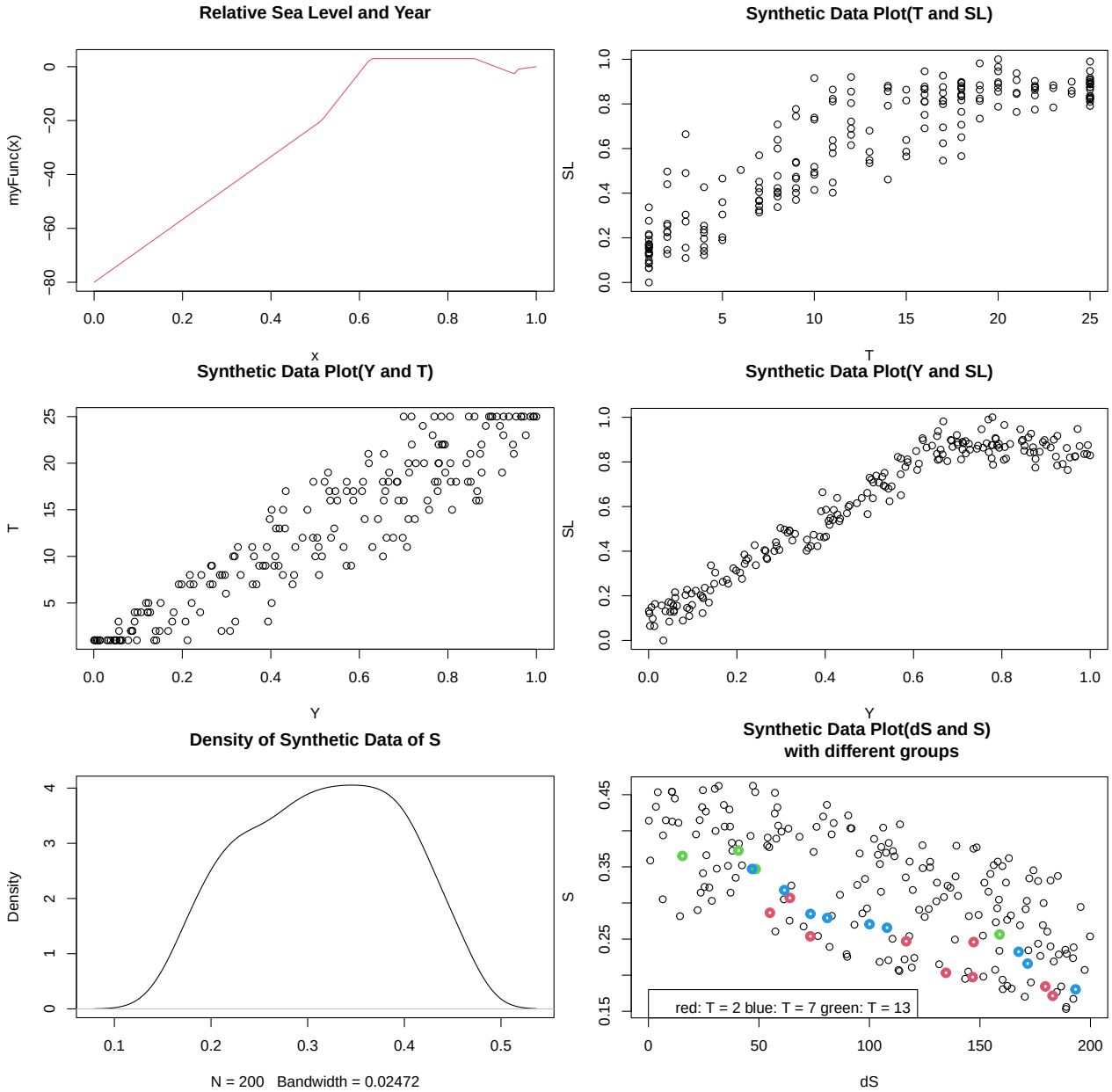
```
## [1] 0.1532178 0.4623421
```

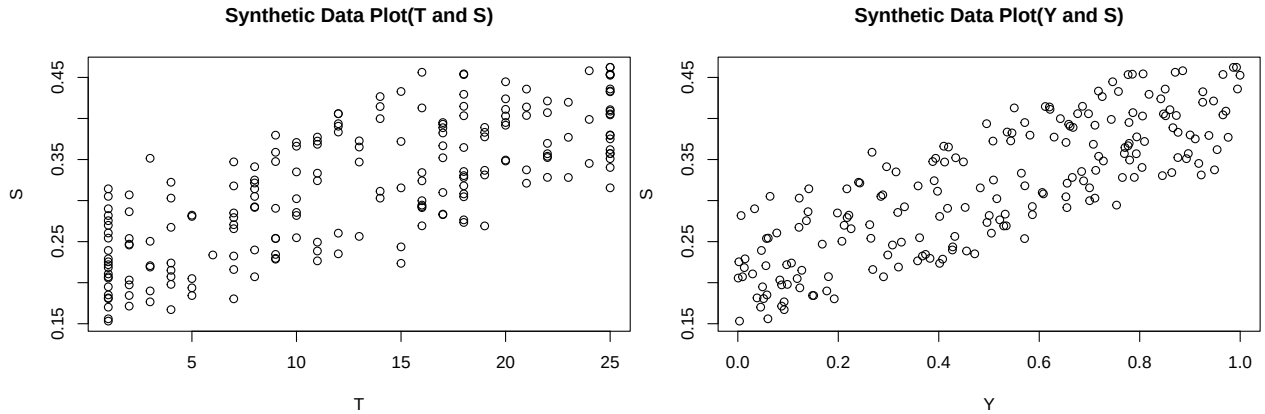
```
plot(density(S), main = "Density of Synthetic Data of S")

plot(dS, S, main = "Synthetic Data Plot(dS and S) \n with different groups")
legend(x = 0, y = 0.18, legend="red: T = 2 blue: T = 7 green: T = 13")
points(dS[T==2], S[T==2], col = 2, lwd =4)
points(dS[T==13], S[T==13], col = 3, lwd =4)
points(dS[T==7], S[T==7], col = 4, lwd =4)
plot(T, S, main = "Synthetic Data Plot(T and S)")
plot(Y, S, main = "Synthetic Data Plot(Y and S)")
plot(T, dS, main = "Synthetic Data Plot(T and dS)\n No correlation expected.") # no correlation
plot(SL, S, main = "Synthetic Data Plot(SL and S)\n auto-correlation")

# Initialize dK with random numbers
dK<-rbeta(length(T), 1.4,1)*400
# K is affected by SL, Y, S, dK
K<-NULL
for(i in 1:length(T))K[i] <- rnorm(1, 1 * SL[i] + 1 * Y[i] + 1 * S[i]- dK[i]/250, 0.2)/10 + 0.2

# Check the data
range(K)# try to make less than 0.5
```

```
## [1] 0.06556851 0.39925771
```

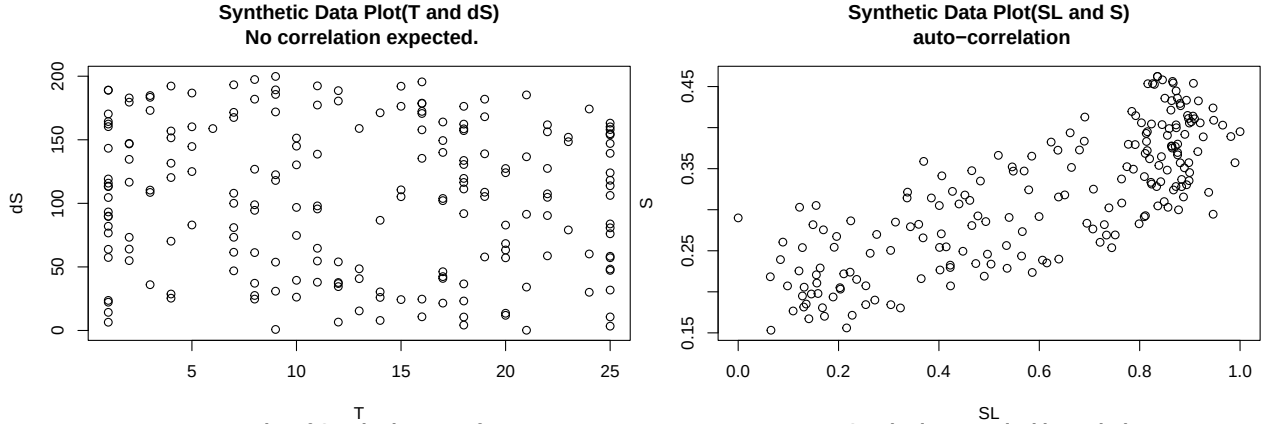
```
plot(density(K), main = "Density of Synthetic Data of K")
plot(dK, K, main = "Synthetic Data Plot(dK and K)")
plot(SL, K, main = "Synthetic Data Plot(SL and K)")
plot(Y, K, main = "Synthetic Data Plot(Y and K)")
plot(S, K, main = "Synthetic Data Plot(S and K)")
plot(dS, K, main = "Synthetic Data Plot(dS and K)\n No correlation expected.") # no correlation

# Standardize data without SL, S, K (those are ratio)
```

```r
d<-list(T=T, dS=standardize(dS), S=S, K=K, dK=standardize(dK))

# check the data
plot(density(d$K), main = "Density of Synthetic Data of d$K")
plot(d$dK, d$K, main = "Synthetic Data Plot(d$dK and d$K)")
plot(SL, d$K, main = "Synthetic Data Plot(SL and d$K)")
plot(Y, d$K, main = "Synthetic Data Plot(Y and d$K)")
plot(d$T, d$K, main = "Synthetic Data Plot(d$T and d$K)")
plot(d$S, d$K, main = "Synthetic Data Plot(d$S and d$K)")
plot(d$dS, d$S, main = "Synthetic Data Plot(d$dS and d$S)")
plot(Y, d$S, main = "Synthetic Data Plot(Y and d$S)")
plot(d$dS, d$K, main = "Synthetic Data Plot(d$dS and d$K)\n No correlation expected.") # no correlation
```

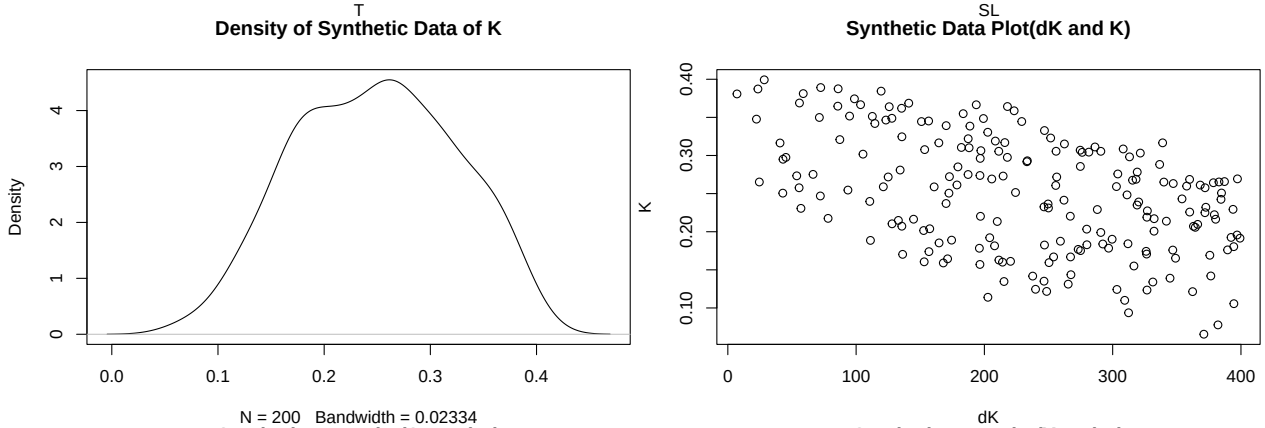



Synthetic Data Plot(T and S)
S
T

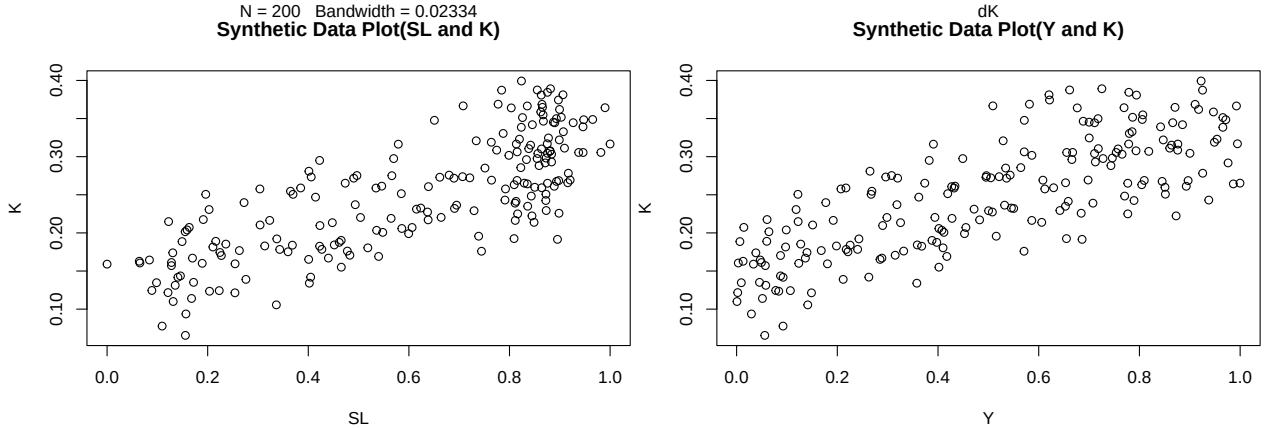
Synthetic Data Plot(Y and S)
S
Y


Synthetic Data Plot(T and dS)
No correlation expected.
dS
T

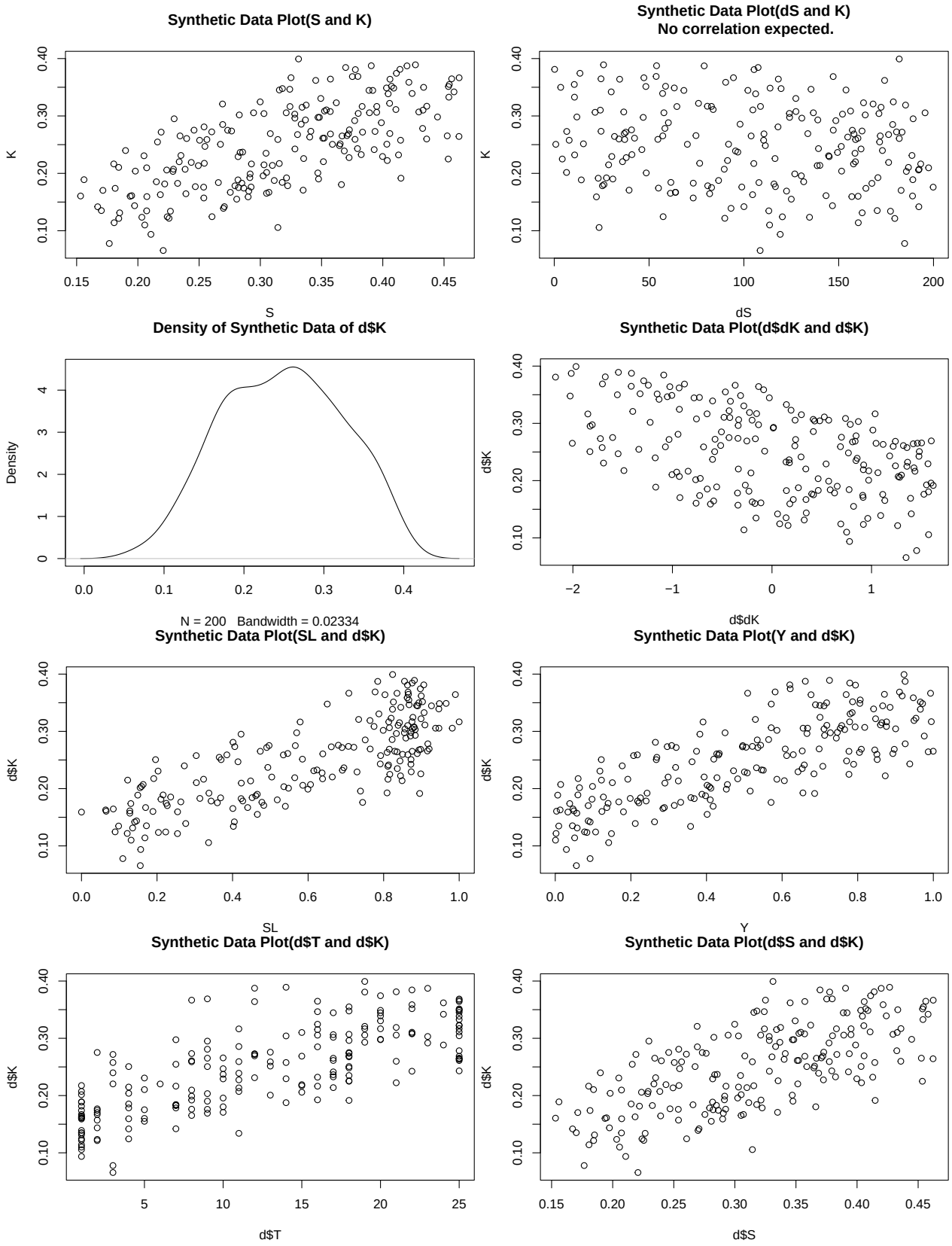
Synthetic Data Plot(SL and S)
auto−correlation
S
SL

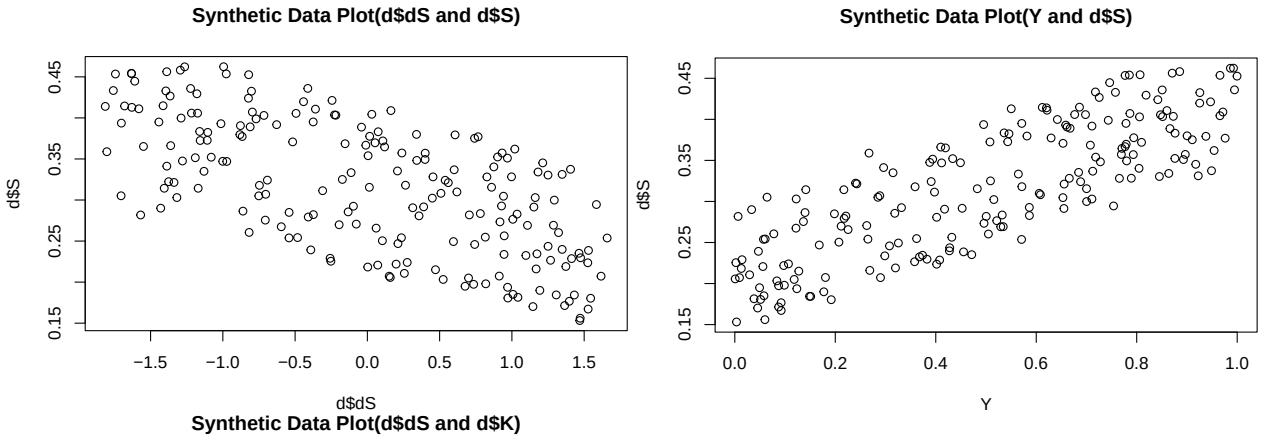

Density of Synthetic Data of K
Density
N = 200 Bandwidth = 0.02334

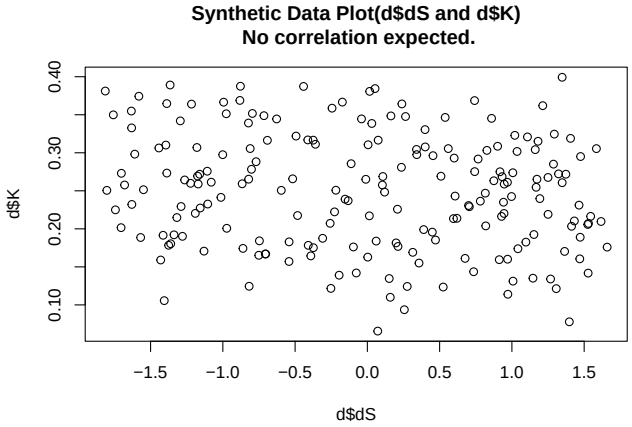
Synthetic Data Plot(dK and K)
K
dK


Synthetic Data Plot(SL and K)
K
SL

Synthetic Data Plot(Y and K)
K
Y


Synthetic Data Plot(S and K)
K
S
Synthetic Data Plot(dS and K)
No correlation expected.
K
dS
Density of Synthetic Data of d$K
Density
N = 200 Bandwidth = 0.02334
Synthetic Data Plot(d$dK and d$K)
d$K
d$dK
Synthetic Data Plot(SL and d$K)
d$K
SL
Synthetic Data Plot(Y and d$K)
d$K
Y
Synthetic Data Plot(d$T and d$K)
d$K
d$T
Synthetic Data Plot(d$S and d$K)
d$K
d$S

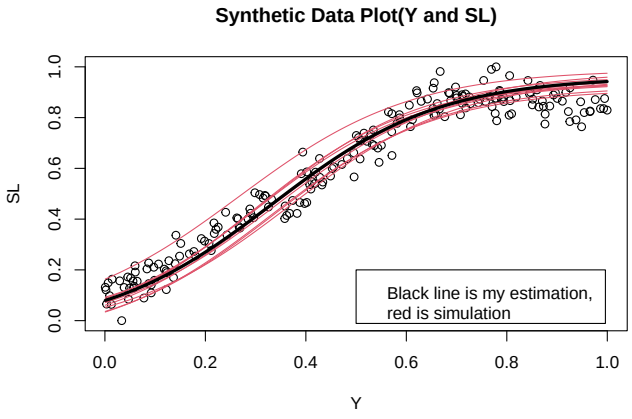




Prior Prediction

- Now I want to create a prior and statistical model.
- As in the overview, the Y and Sl are hidden parameter so how to initialize is important
- This time, to make it simple, I use logit function to immitate the *myFunction*

```
plot(Y, SL, main = "Synthetic Data Plot(Y and SL)")
# Simulation
for(i in 1:10)curve(inv_logit(rnorm(1, 6, 0.2)*x + rnorm(1, -2, 0.2)) - rnorm(1, 0.04, 0.02), add =TRUE
curve(inv_logit(6*x - 2) - 0.04, add =TRUE, lwd = 3)
legend(0.5, 0.2,legend="Black line is my estimation, \nred is simulation")
```



Model 1

- This is multi-model of the DAG with out any stratification See Overview pdf for details.
- There are a few messages to warning my priors. Since it only shows up during first iteration of warmup, we can safely ignore it.
- Warnings are annoying but relatively few so I just omit it for m1.2 and m1.3.

```
m1<- ulam(
    alist (
    # Main model K
        K ~ normal(mu_K, U),
        logit(mu_K) <- aK + bK_S * S + bK_SL * SL[i] + bK_Y * Y[i] + bK_dK * dK + bK_T[T],
        # We do not know anything about this but think similar for all group.
        # Mean should start at 0 and has small variation
        # bK_T is regularized to maintain the efficiency.

        c(aK, bK_S,  bK_SL, bK_Y, bK_dK) ~ normal(0,0.1), # This is different from the other one, so th
        vector[25]: bK_T <- bK_T_bar + sigma_bK_T * z_bK_T[T],
        bK_T_bar ~ normal(0, 0.1), # regularization but not high sigma
        sigma_bK_T ~ exponential(1), # Needs to be 1 to maintain bK_T.
        z_bK_T[T] ~ normal(0, 0.1),

        U ~ exponential(1),

    # Model S
        S ~ normal(mu_S, sigma_S),
        logit(mu_S) <- aS + bS_dS * dS + bS_Y * Y[i] + bS_T[T],

        # I am hoping/believing b_dS is negative, b_Y_S is positive, and bS_T is different for each one
        c(aS, bS_Y, bS_dS) ~ normal(0,0.1),
        # bS_T is regularized.
        vector[25]: bS_T <- bS_T_bar + sigma_bS_T * z_bS_T[T],
        bS_T_bar ~ normal(0, 0.1),
        sigma_bS_T ~ exponential(1),
        z_bS_T[T] ~ normal(0, 0.1),

        sigma_S ~ exponential(1),

    # Model SL by Y
        # The log function and prior was refered to the previous research of Sea Level Change over time
        # This function is abstruct so I will have to actually stan code to provide accurate estimates
        # ulam code in r does not allow us to hartd code the conditional function.
        # vector[200]: SL ~ normal(muSL + aSL2, sigmaSL),
        vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

    # Model T, This should be hard coded based on the previous research
        T ~ poisson(Y * 25),

    # Model T by Y
        vector[200]:Y ~ normal(0.5, 0.3)

    ), dat = d, constraints = list(
        SL    = "lower=0,upper=1", # SL is between 0 and 1
        Y    = "lower=0,upper=1" # Y is between 0 and 1
        #, T    = "lower=1,upper=25"  #this is unnecessary since T is provided in the dat
```

```
    ), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(
    adapt_delta = 0.99
  )
)
```

```
## In file included from stan/src/stan/model/model_header.hpp:5:
## stan/src/stan/model/model_base_crtp.hpp:205:8: warning: 'void stan::model::model_base_crtp<M>::write
##   205 |   void write_array(stan::rng_t& rng, std::vector<double>& theta,
##       |        ^~~~~~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpGKTkkN/model-c3bc2d2075d1.hpp:1308:3: note:   by 'void ulam_cm
##  1308 |   write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
##       |   ^~~~~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:136:8: warning: 'void stan::model::model_base_crtp<M>::write
##   136 |   void write_array(stan::rng_t& rng, Eigen::VectorXd& theta,
##       |        ^~~~~~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpGKTkkN/model-c3bc2d2075d1.hpp:1308:3: note:   by 'void ulam_cm
##  1308 |   write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
##       |   ^~~~~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:96:20: warning: 'stan::math::var stan::model::model_base_crt
##    96 |   inline math::var log_prob(Eigen::Matrix<math::var, -1, 1>& theta,
##       |                    ^~~~~~~
## C:/Use
## rs/nobut/AppData/Local/Temp/RtmpGKTkkN/model-c3bc2d2075d1.hpp:1345:3: note:   by 'T_ ulam_cmdstanr_0
##  1345 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:154:17: warning: 'double stan::model::model_base_crtp<M>::log
##   154 |   inline double log_prob(std::vector<double>& theta, std::vector<int>& theta_i,
##       |                 ^~~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpGKTkkN/model-c3bc2d2075d1.hpp:1345:3: note:   by 'T_ ulam_cmds
##  1345 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:91:17: warning: 'double stan::model::model_base_crtp<M>::log
##    91 |   inline double log_prob(Eigen::VectorXd& theta,
##       |                 ^~~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpGKTkkN/model-c3bc2d2075d1.hpp:1345:3: note:   by 'T_ ulam_cmds
## const'
##  1345 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:159:20: warning: 'stan::math::var stan::model::model_base_cr
##   159 |   inline math::var log_prob(std::vector<math::var>& theta,
##       |                    ^~~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpGKTkkN/model-c3bc2d2075d1.hpp:1345:3: note:   by 'T_ ulam_cmds
##  1345 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of th
## Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppDa
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like cova
```

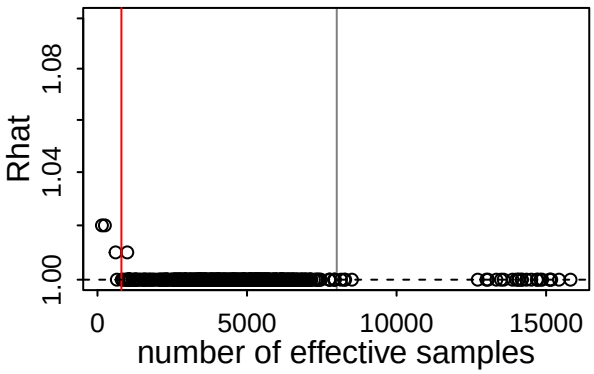
```
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 1
## Chain 2 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 2 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppDa
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 2
## Chain 3 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppDa
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 3
## Chain 4 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 1 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 2 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 3 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 4 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 1 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 1 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 2 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 4 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 1 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 3 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 2 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 2 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 1 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 4 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 1 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 4 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 3 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 2 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 1 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 2 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 3 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 4 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 1 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 2 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 4 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 3 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 2 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 4 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 1 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 3 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 4 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%]  (Warmup)
```

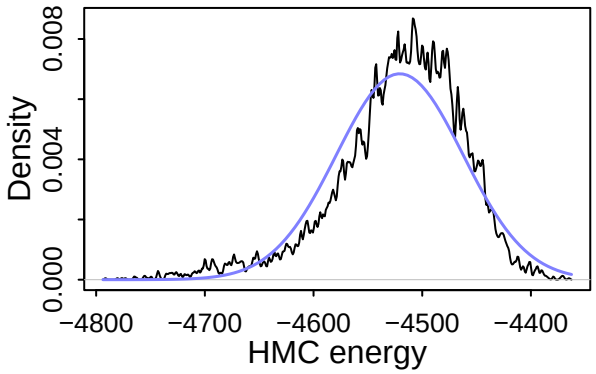
```
## Chain 3 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 2 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 4 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 3 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 3 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 1 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 2 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 4 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 3 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 1 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 3 Iteration: 2000 / 4000 [ 50%]  (Warmup)
```

```
## Chain 3 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%]  (Sampling)
```

```
## Chain 3 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 4 finished in 221.4 seconds.
## Chain 2 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 2 finished in 231.6 seconds.
## Chain 3 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 3 finished in 232.5 seconds.
## Chain 1 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 1 finished in 278.7 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 241.0 seconds.
## Total execution time: 278.8 seconds.

## Warning: 4 of 4 chains had an E-BFMI less than 0.3.
## See https://mc-stan.org/misc/warnings for details.
```

```
dashboard(m1)
```

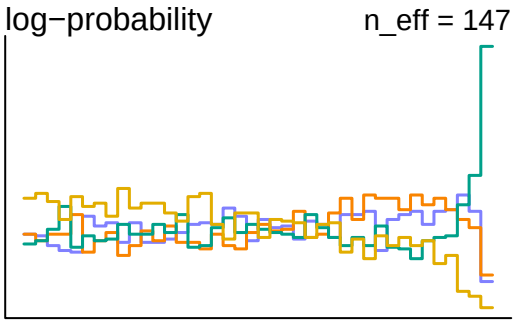




0

Divergent transitions

Outlook good



- without divergent but it has a few problem with efficiency, as I can see (might be different on what you see due to randomized data).
- I will try to eliminate by removing parameters.

```
m1.2<- ulam(
    alist (
    # Main model K
        K ~ normal(mu_K, U),
        logit(mu_K) <- aK + bK_S * S + bK_SL * SL[i] + bK_dK * dK + bK_T[T],

        c(aK, bK_S,  bK_SL, bK_Y, bK_dK) ~ normal(0,0.1),
        vector[25]: bK_T <- bK_T_bar + sigma_bK_T * z_bK_T[T],
        bK_T_bar ~ normal(0, 0.1),
        sigma_bK_T ~ exponential(1),
        z_bK_T[T] ~ normal(0, 0.1),

        U ~ exponential(1),

    # Model S
        S ~ normal(mu_S, sigma_S),
        logit(mu_S) <- aS + bS_dS * dS + bS_Y * Y[i] + bS_T[T],

        c(aS, bS_Y, bS_dS) ~ normal(0,0.1),
        vector[25]: bS_T <- bS_T_bar + sigma_bS_T * z_bS_T[T],
        bS_T_bar ~ normal(0, 0.1),
        sigma_bS_T ~ exponential(1),
        z_bS_T[T] ~ normal(0, 0.1),

        sigma_S ~ exponential(1),

    # Model SL by Y
```

```
        vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

    # Model T
        T ~ poisson(Y * 25),

    # Model T by Y
        vector[200]:Y ~ normal( 0.5, 0.2)
    ), dat = d, constraints = list(
        SL   = "lower=0,upper=1",
        Y    = "lower=0,upper=1"
    ), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(
    adapt_delta = 0.99
  )
)
```

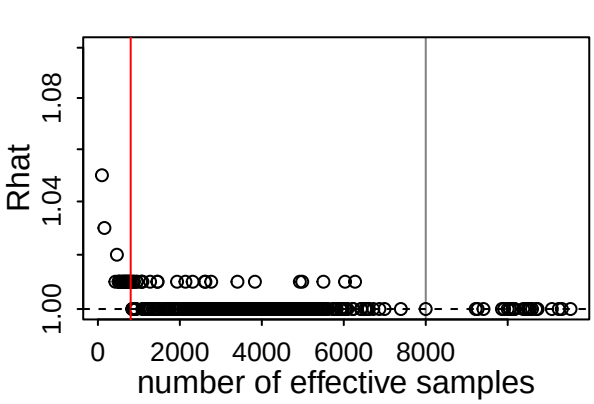
```
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 3 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 4 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 2 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 1 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 2 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 4 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 3 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 3 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 2 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 4 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 1 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 3 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 2 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 4 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 1 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 3 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 1 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 3 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 4 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 2 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 1 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 3 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 4 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 2 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 1 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 2 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 4 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 1 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 3 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 2 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 4 Iteration:  800 / 4000 [ 20%]  (Warmup)
```

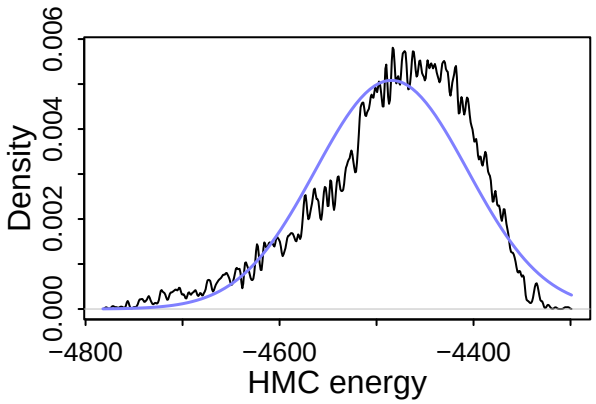
```
## Chain 1 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 4 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 1 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 3 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 2 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 2 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 3 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 1 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 4 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 3 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 4 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%]  (Sampling)
```

```
## Chain 1 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 3 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 2 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 3 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 2 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 3 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 2 finished in 211.5 seconds.
## Chain 4 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%]  (Sampling)
```

```
## Chain 4 finished in 264.3 seconds.
## Chain 3 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 1 finished in 374.6 seconds.
## Chain 3 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 3 finished in 439.7 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 322.5 seconds.
## Total execution time: 439.9 seconds.
```

```
dashboard(m1.2)
```

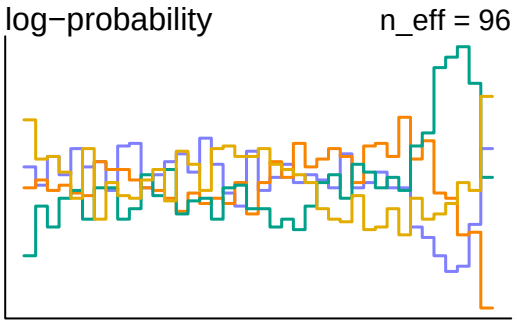




8

Divergent transitions

Roll again



- This is still not the best model with low efficiency. I will remove one more parameter.
- I can remove bK_T or bS_T but this time, I want to keep the information as other models keep it by stratification.

```
m1.3<- ulam(
    alist (
    # Main model K
        K ~ normal(mu_K, U),
        logit(mu_K) <- aK + bK_S * S + bK_SL * SL[i] + bK_dK * dK + bK_T[T],

        c(aK, bK_S,  bK_SL, bK_dK) ~ normal(0,0.1),
        vector[25]: bK_T <- bK_T_bar + sigma_bK_T * z_bK_T[T],
        bK_T_bar ~ normal(0, 0.1),
        sigma_bK_T ~ exponential(1),
        z_bK_T[T] ~ normal(0, 0.1),

        U ~ exponential(1),

    # Model S
        S ~ normal(mu_S, sigma_S),
        logit(mu_S) <- aS + bS_dS * dS + bS_T[T],

        c(aS, bS_dS) ~ normal(0,0.1),
        vector[25]: bS_T <- bS_T_bar + sigma_bS_T * z_bS_T[T],
        bS_T_bar ~ normal(0, 0.1),
        sigma_bS_T ~ exponential(1),
        z_bS_T[T] ~ normal(0, 0.1),

        sigma_S ~ exponential(1),
```

```
    # Model SL by Y
        vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

    # Model T
        T ~ poisson(Y * 25),

    # Model T by Y
        vector[200]:Y ~ normal( 0.5, 0.2)
    ), dat = d, constraints = list(
        SL   = "lower=0,upper=1",
        Y    = "lower=0,upper=1"
    ), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(
    adapt_delta = 0.99
  )
)
```

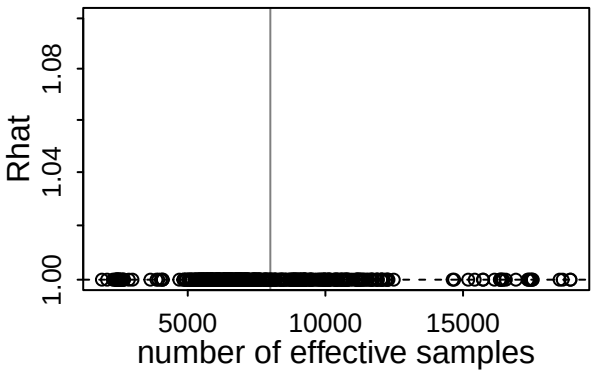
```
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 1 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 2 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 4 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 1 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 4 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 3 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 2 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 1 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 4 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 3 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 2 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 1 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 4 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 2 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 3 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 4 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 1 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 2 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 3 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 4 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 1 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 3 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 2 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 4 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 1 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 3 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 2 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 4 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 1 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 3 Iteration:  700 / 4000 [ 17%]  (Warmup)
```

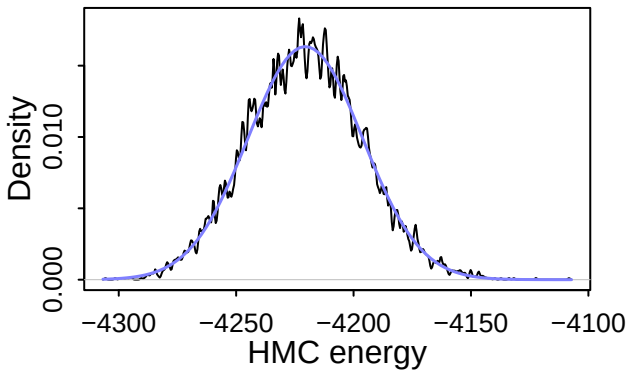
```
## Chain 2 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 4 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 3 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 1 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 2 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 3 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 4 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 2 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 3 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 4 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 3 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%]  (Sampling)
```

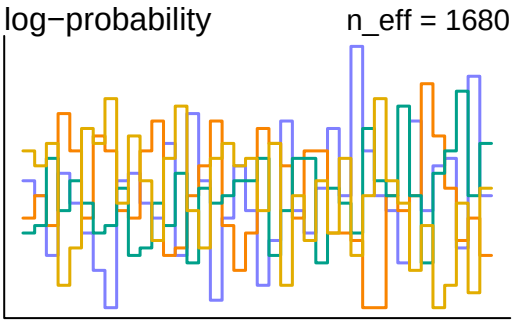
```
## Chain 2 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 4 finished in 94.2 seconds.
## Chain 1 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%]  (Sampling)
```

```
## Chain 1 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 1 finished in 116.7 seconds.
## Chain 2 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 2 finished in 131.2 seconds.
## Chain 3 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 3 finished in 131.9 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 118.5 seconds.
## Total execution time: 132.0 seconds.
```

```
dashboard(m1.3)
```








```
precis(m1.3, depth = 1)
```

```
##                   mean          sd        5.5%       94.5%     rhat  ess_bulk
## bK_dK       -0.19858589 0.013074730 -0.21937215 -0.17779064 1.000274 14677.031
## bK_SL        0.35570330 0.095248288  0.20625417  0.50570237 1.000826  3644.342
## bK_S         0.20652863 0.096427873  0.05057434  0.35894034 1.000561 11283.313
## aK          -0.21185100 0.108576295 -0.38588665 -0.04071583 1.001106  5801.541
## bK_T_bar    -0.21245762 0.109716030 -0.39086547 -0.04000755 1.000543  5501.787
## sigma_bK_T   8.25838037 1.765169581  5.61457663 11.20213569 1.001049  2495.811
## U            0.03256399 0.002299381  0.02898347  0.03626199 1.000370  4087.679
## bS_dS       -0.21918332 0.011371851 -0.23737037 -0.20080157 1.000412 17379.852
## aS          -0.30023146 0.085383694 -0.43490553 -0.16200147 1.000418  5357.890
## bS_T_bar    -0.30134377 0.085530857 -0.43502895 -0.16296834 1.000387  6028.057
## sigma_bS_T   2.83277308 0.761731229  1.87151942  4.19933392 1.000904  1883.543
## sigma_S      0.03300702 0.001737829  0.03037994  0.03584091 1.001045 11144.199
```

- This looks much better than previous ones with very high eff value. ### Posterior Check

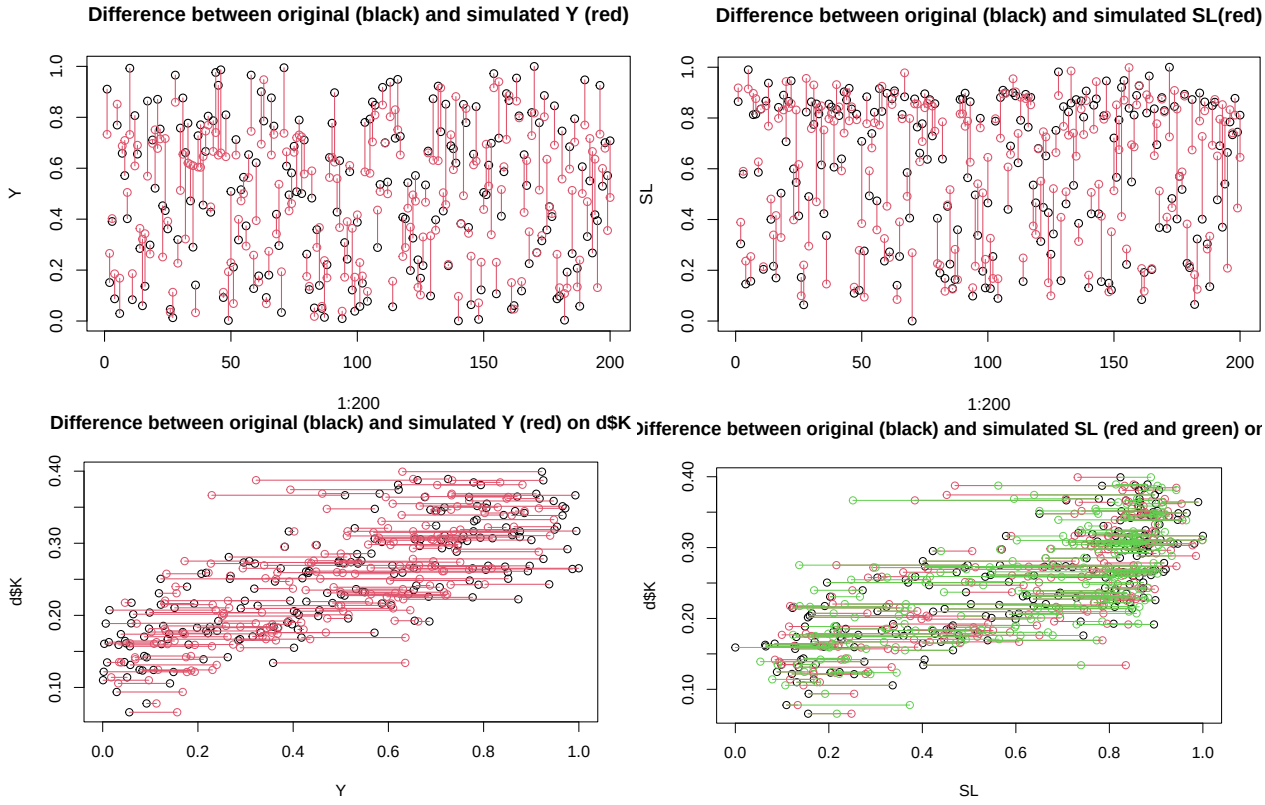
```
post <- extract.samples(m1.3)
```

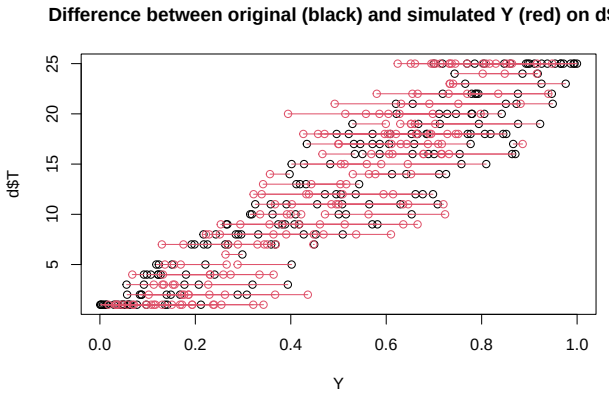
Hidden parameters

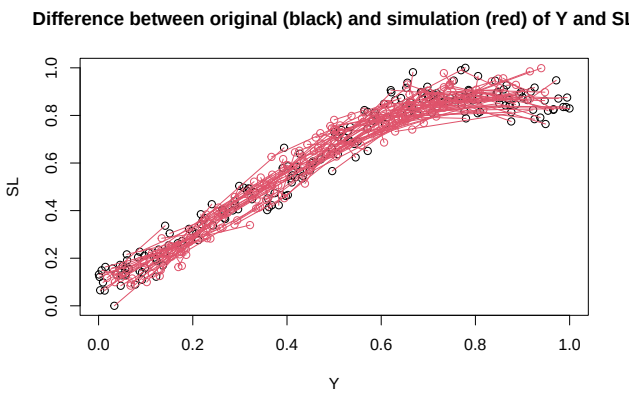
- The hidden parameters are estimated by the model constraints and distributions.
- It is expected that the difference is small and has similar trend as the original data.

```
# Posterior Y changes
plot(1:200, Y, main = "Difference between original (black) and simulated Y (red)")
points(1:200, post$Y[1,], col = 2)
for(i in 1 : 200) lines(c(i, i), c(post$Y[1,i], Y[i]), col = 2)

plot(1:200, SL, main = "Difference between original (black) and simulated SL(red)")
points(1:200, post$SL[1,], col = 2)
```

```r
for(i in 1 : 200) lines(c(i, i), c(post$SL[1,i], SL[i]), col = 2)

# check Y and SL with K
plot(Y, d$K, main = "Difference between original (black) and simulated Y (red) on d$K")
points(post$Y[1,], d$K, col = 2)
for(i in 1 : 200) lines(c(post$Y[1,i], Y[i]), c(d$K[i], d$K[i]), col = 2)

plot(SL, d$K, main = "Difference between original (black) and simulated SL (red and green) on d$K")
points(post$SL[1,], d$K, col = 2)
for(i in 1 : 200) lines(c(post$SL[1,i], SL[i]), c(d$K[i], d$K[i]), col = 2)
points(post$SL[2,], d$K, col = 3)
for(i in 1 : 200) lines(c(post$SL[2,i], SL[i]), c(d$K[i], d$K[i]), col = 3)

# Y and T
plot(Y, d$T, main = "Difference between original (black) and simulated Y (red) on d$T")
points(post$Y[1,], d$T ,col = 2)
for (i in 1:200) lines(c(Y[i], post$Y[1,i]), c(d$T[i],d$T[i]), col = 2)

# Y and SL both hidden parameters
plot(Y, SL, main = "Difference between original (black) and simulation (red) of Y and SL")
points(post$Y[1,], post$SL[1,], col=2)
for(i in 1 : 200) lines(c(Y[i], post$Y[1,i]), c(SL[i], post$SL[1,i]), col = 2)
```







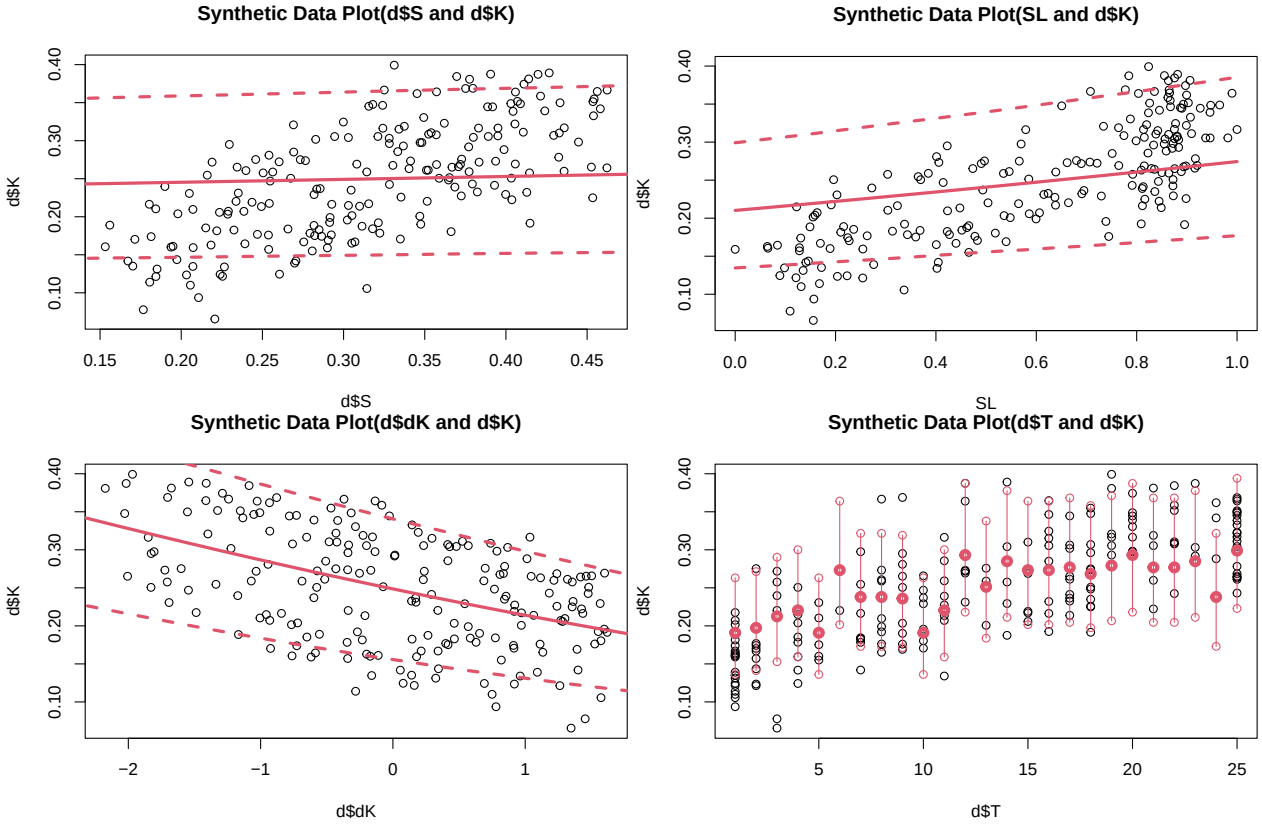
=> this shows that the difference is not very large and hidden parameters have similar trend as the original (synthetized) data.

Posterior Simulation

- Let us take a look at the Posterior simulation, if simulated data align with original data.
- The red solid line is the mean and dashed lines are 95% percentile interval. if the most of datpoints are inside the percentile interval, the simulated data is very simular to the original data. ##### Model K

```
plot(d$S, d$K, main = "Synthetic Data Plot(d$S and d$K)")
Sseq<-seq(0, 0.5, len = 20)
KbyS<- sapply(Sseq,
    function (i) inv_logit(post$aK + post$bK_S * i + post$bK_SL * colMeans(post$SL) + post$bK_dK * d$dK
means<- apply( KbyS , 2 , mean )
PIs<- apply( KbyS , 2 , PI )
lines(Sseq, means, col = 2, lwd = 3)
lines(Sseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(Sseq, PIs[2, ], col = 2, lty = 2, lwd = 3)


plot(SL, d$K, main = "Synthetic Data Plot(SL and d$K)")
SLseq<-seq(0, 1, len = 20)
KbySL<- sapply(SLseq,
    function (i) inv_logit(post$aK + post$bK_S * d$S + post$bK_SL * i + post$bK_dK * d$dK + colMeans(pos
means<- apply( KbySL , 2 , mean )
PIs<- apply( KbySL , 2 , PI )
lines(SLseq, means, col = 2, lwd = 3)
lines(SLseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(SLseq, PIs[2, ], col = 2, lty = 2, lwd = 3)


plot(d$dK, d$K, main = "Synthetic Data Plot(d$dK and d$K)")
dKseq<-seq(-2.5, 2, len = 20)
KbydK<- sapply(dKseq,
    function (i) inv_logit(post$aK + post$bK_S * d$S + post$bK_SL * colMeans(post$SL) + post$bK_dK * i
PIs<- apply( KbydK , 2 , PI )
means<- apply( KbydK , 2 , mean )
lines(dKseq, means, col = 2, lwd = 3)
lines(dKseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dKseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(d$T, d$K, main = "Synthetic Data Plot(d$T and d$K)")
```

```
KbyT<- sapply(1:25,
    function (i) inv_logit(post$aK + post$bK_S * d$S + post$bK_SL * colMeans(post$SL) + post$bK_dK * d$
PIs<- apply( KbyT , 2 , PI )
means<- apply( KbyT , 2 , mean )
points(1:25, means, col = 2, lwd = 4)
points(1:25, PIs[1, ], col = 2, lty = 2 )
points(1:25, PIs[2, ], col = 2, lty = 2 )
for(i in 1:25) lines(c(i,i), c(PIs[1, i], PIs[2, i]), col = 2)
```



- Looks nice as most of the original data inside 95% Percentile Interval!

Model S

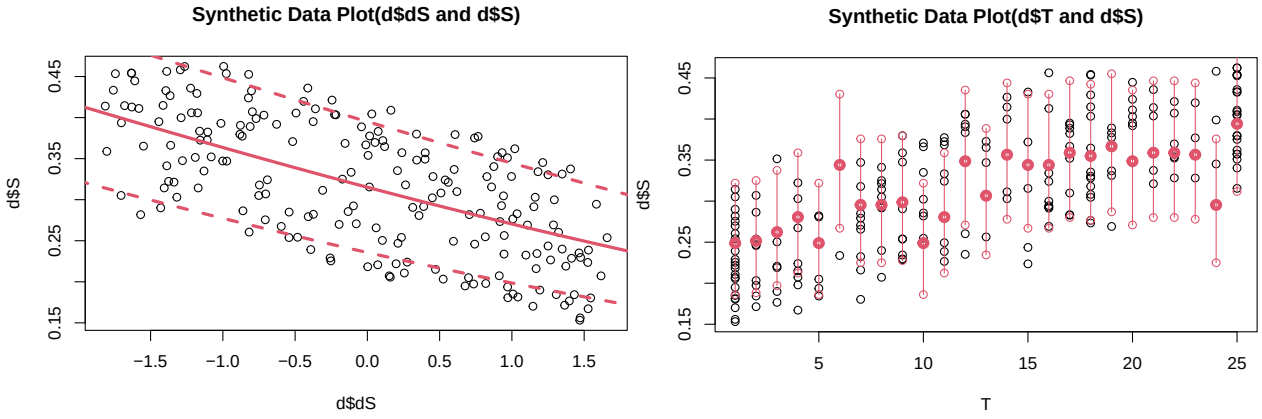
- Do the same thing for S
- By the way, there is no post-S exists as it seems absolutely same as post-SL
- would be because it is one of the predictors that i provided in the data.

```
post$SL[1,] == post$S[1,]
```

```
##   [1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
##  [16] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
##  [31] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
##  [46] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
##  [61] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
##  [76] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
##  [91] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [106] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [121] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [136] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
```

```
## [151] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [166] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [181] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [196] TRUE TRUE TRUE TRUE TRUE
```

```r
plot(d$dS, d$S, main = "Synthetic Data Plot(d$dS and d$S)")
dSseq<-seq(-2, 2, len = 20)
SbydS<- sapply(dSseq,
    function (i) inv_logit(post$aS + post$bS_dS * i + colMeans(post$bS_T)[d$T] )) # Used mean of each s
PIs<- apply( SbydS , 2 , PI )
means<- apply( SbydS , 2 , mean )
lines(dSseq, means, col = 2, lwd = 3)
lines(dSseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dSseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(T, d$S, main = "Synthetic Data Plot(d$T and d$S)")
SbyT<- sapply(1:25,
    function (i) inv_logit(post$aS + post$bS_dS * d$dS + colMeans(post$bS_T)[i]))
PIs<- apply( SbyT , 2 , PI )
means<- apply( SbyT , 2 , mean )
points(1:25, means, col = 2, lwd = 4)
points(1:25, PIs[1, ], col = 2, lty = 2)
points(1:25, PIs[2, ], col = 2, lty = 2)
for(i in 1:25) lines(c(i,i), c(PIs[1, i], PIs[2, i]), col = 2)
```



This looks great.

Model 2

- This model stratifies all parameters by T.
- Since T is categorical parameter, it is easy to group by T.
- Also see overview.pdf to know statistical model and prior

```
# Model 2
m2<- ulam(
    alist (
    # Main model K
        K ~ normal(mu_K, U),
        logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_Y[T] * Y[i] + bK_dK * dK,

        # Basically same as m1 but regularization (partial pooling).
        # This does not need correlation because multilevel models of S and SL includes the information
        # But we have to regularize
        vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
        vector [25]: bK_S <- bK_S_bar + sigma_bK_S * z_bK_S[T],
        vector [25]: bK_SL <- bK_SL_bar + sigma_bK_SL * z_bK_SL[T],
        vector [25]: bK_Y <- bK_Y_bar + sigma_bK_Y * z_bK_Y[T],
        c(aK_bar, bK_S_bar,  bK_SL_bar, bK_Y_bar) ~ normal(0,0.3),
        c(sigma_aK, sigma_bK_S, sigma_bK_SL, sigma_bK_Y) ~ exponential(6), # want low variation among g
        z_aK[T] ~ normal(0,0.1),
        z_bK_S[T] ~ normal(0,0.1),
        z_bK_SL[T] ~ normal(0,0.1),
        z_bK_Y[T] ~ normal(0,0.1),

        # don't need regularization as dK is not affected by T
        bK_dK ~ normal(0, 0.3),
         U ~ exponential(1),

     # Model S
         S ~ normal(mu_S, sigma_S),
        logit(mu_S) <- aS[T] + bS_Y[T] * Y[i] + bS_dS * dS,

         # Belows are similar to m1.
         # partial pooling for the same reason for a2 and b_Y_S
         vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
         vector[25]: bS_Y<-bS_Y_bar + sigmabS_Y * zbS_Y[T],
         c(aS_bar, bS_Y_bar) ~ normal(0,0.3),
         c(sigma_aS, sigmabS_Y) ~ exponential(6),
         z_aS[T] ~ normal(0,0.1),
         zbS_Y[T] ~ normal(0,0.1),

        # don't need regularization
         bS_dS ~ normal(0,0.3),
         sigma_S ~ exponential(1),

    # Model SL by Y
        vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

    # Model T, This should be hard coded based on the previous research
        T ~ poisson(Y * 25),

    # Model T by Y
```

```
        vector[200]:Y ~ normal(0.5, 0.3)

    ), dat = d, constraints = list(
        SL    = "lower=0,upper=1",
        Y     = "lower=0,upper=1"
    ), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE
)
```

```
## In file included from stan/src/stan/model/model_header.hpp:5:
## stan/src/stan/model/model_base_crtp.hpp:205:8: warning: 'void stan::model::model_base_crtp<M>::write
##   205 |   void write_array(stan::rng_t& rng, std::vector<double>& theta,
##       |        ^~~~~~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpGKTkkN/model-c3bc3ecf1bf.hpp:1843:3: note:   by 'void ulam_cmds
##  1843 |   write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
##       |   ^~~~~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:136:8: warning: 'void stan::model::model_base_crtp<M>::write
##   136 |   void write_array(stan::rng_t& rng, Eigen::VectorXd& theta,
##       |        ^~~~~~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpGKTkkN/model-c3bc3ecf1bf.hpp:1843:3: note:   by 'void ulam_cmds
##  1843 |   write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
##       |   ^~~~~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:96:20: warning: 'stan::math::var stan::model::model_base_crtp
##    96 |   inline math::var log_prob(Eigen::Matrix<math::var, -1, 1>& theta,
##       |                    ^~~~~~~
## C:/Users
## /nobut/AppData/Local/Temp/RtmpGKTkkN/model-c3bc3ecf1bf.hpp:1880:3: note:   by 'T_ ulam_cmdstanr_e782
##  1880 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:154:17: warning: 'double stan::model::model_base_crtp<M>::log
##   154 |   inline double log_prob(std::vector<double>& theta, std::vector<int>& theta_i,
##       |                 ^~~~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpGKTkkN/model-c3bc3ecf1bf.hpp:1880:3: note:   by 'T_ ulam_cmdsta
##  1880 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:91:17: warning: 'double stan::model::model_base_crtp<M>::log_
##    91 |   inline double log_prob(Eigen::VectorXd& theta,
##       |                 ^~~~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpGKTkkN/model-c3bc3ecf1bf.hpp:1880:3: note:   by 'T_ ulam_cmdsta
## '
##  1880 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:159:20: warning: 'stan::math::var stan::model::model_base_cr
##   159 |   inline math::var log_prob(std::vector<math::var>& theta,
##       |                    ^~~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpGKTkkN/model-c3bc3ecf1bf.hpp:1880:3: note:   by 'T_ ulam_cmdsta
##  1880 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~~
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
```

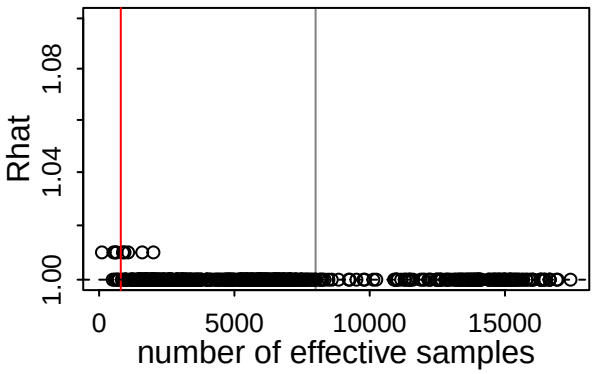
```
## Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppDa
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 1
## Chain 2 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 2 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppDa
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 2
## Chain 3 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppDa
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 3
## Chain 4 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppDa
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 4
## Chain 1 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 2 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 3 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 4 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 2 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 4 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 1 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 2 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 4 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 3 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 2 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 4 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 3 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 2 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 4 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 3 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 1 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 2 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 4 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 3 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 1 Iteration:  400 / 4000 [ 10%]  (Warmup)
```

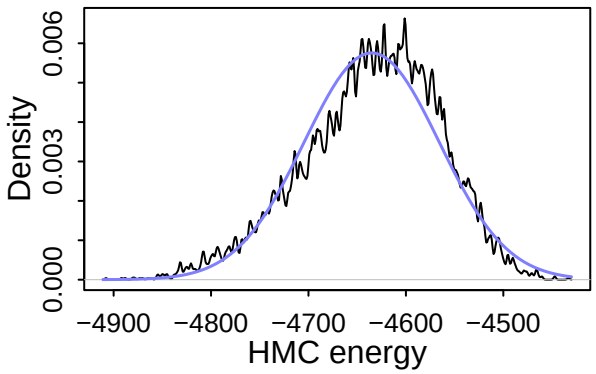
```
## Chain 2 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 4 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 3 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 4 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 2 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 1 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 3 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 2 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 4 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 1 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 3 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 3 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 1 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 1 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 1 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 4 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 1 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 2 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%]  (Sampling)
```

```
## Chain 3 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 4 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 1 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 2 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 3 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 1 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 3 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 1 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 3 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 1 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 3 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%]  (Sampling)
```

```
## Chain 2 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 2 finished in 54.2 seconds.
## Chain 4 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 4 finished in 54.8 seconds.
## Chain 1 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 3 finished in 56.2 seconds.
## Chain 1 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 1 finished in 61.9 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 56.8 seconds.
## Total execution time: 62.0 seconds.

## Warning: 10 of 8000 (0.0%) transitions ended with a divergence.
## See https://mc-stan.org/misc/warnings for details.

## Warning: 4 of 4 chains had an E-BFMI less than 0.3.
## See https://mc-stan.org/misc/warnings for details.
```

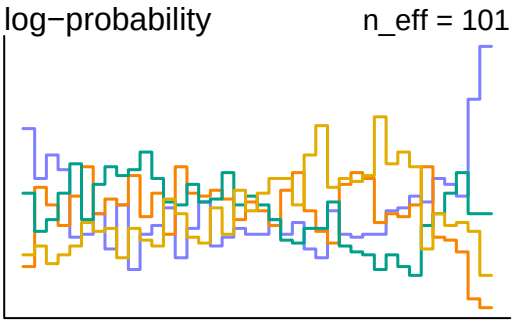
```
dashboard(m2)
```





10
Divergent transitions

Check yourself before you wreck yourself



- This has so much divergent and less effective sampling.
- This would be due to the large number of parameters, including hidden estimated ones (SL and Y). I will reduce parameters to reliably interpret the result.
  - First one would be Y as S and SL are more important for the model.
    - by the way, SL and Y in model K change the efficiency in the highest rate.

```
# Model 2.2
m2.2<- ulam(
    alist (
    # Main model K
        K ~ normal(mu_K, U),
        logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_dK * dK,
        vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
        vector [25]: bK_S <- bK_S_bar + sigma_bK_S * z_bK_S[T],
        vector [25]: bK_SL <- bK_SL_bar + sigma_bK_SL * z_bK_SL[T],
        c(aK_bar, bK_S_bar,  bK_SL_bar) ~ normal(0,0.3),
        c(sigma_aK, sigma_bK_S, sigma_bK_SL) ~ exponential(6),
        z_aK[T] ~ normal(0,0.1),
        z_bK_S[T] ~ normal(0,0.1),
        z_bK_SL[T] ~ normal(0,0.1),
        bK_dK ~ normal(0, 0.3),
        U ~ exponential(1),

    # Model S
        S ~ normal(mu_S, sigma_S),
        logit(mu_S) <- aS[T] + bS_Y[T] * Y[i] + bS_dS * dS ,
        vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
        vector[25]: bS_Y<-bS_Y_bar + sigmabS_Y * zbS_Y[T],
        c(aS_bar, bS_Y_bar) ~ normal(0,0.3),
        c(sigma_aS, sigmabS_Y) ~ exponential(6),
        z_aS[T] ~ normal(0,0.1),
```

```
        zbS_Y[T] ~ normal(0,0.1),
        bS_dS ~ normal(0,0.3),
        sigma_S ~ exponential(1),

        # Model SL by Y
        vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

        # Model T, This should be hard coded based on the previous research
        T ~ poisson(Y * 25),

        # Model T by Y
        vector[200]:Y ~ normal( 0.5, 0.2)

    ), dat = d, constraints = list(
        SL    = "lower=0,upper=1"
        , Y    = "lower=0,upper=1"
    ), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE
)
```

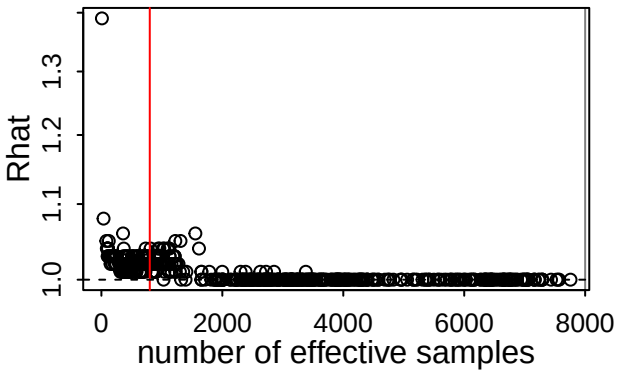
```
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 3 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 1 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 2 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 4 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 3 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 3 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 1 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 2 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 3 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 4 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 1 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 2 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 3 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 2 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 1 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 3 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 3 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 1 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 2 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 3 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 1 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 4 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 3 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 1 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 2 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 1 Iteration:  800 / 4000 [ 20%]  (Warmup)
```

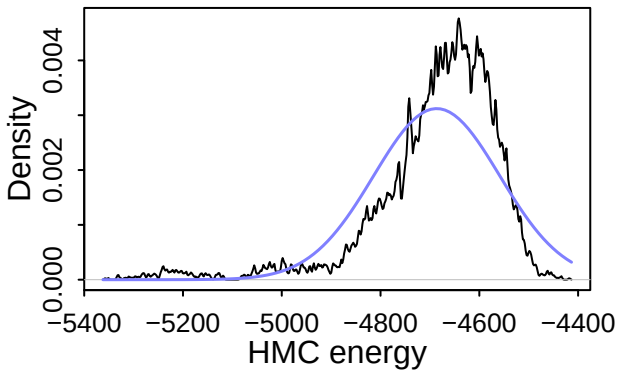
```
## Chain 3 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 1 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 4 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 2 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 4 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 4 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 4 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 3 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 4 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 3 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 4 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 3 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 1 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 2 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 3 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 4 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 3 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 4 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 3 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 1 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 3 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 4 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 3 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 1 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 3 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 1 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 3 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 4 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 3 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 1 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 3 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 2 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 3 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 1 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 3 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 4 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 3 Iteration: 3700 / 4000 [ 92%]  (Sampling)
```

```
## Chain 3 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 4 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 1 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 4 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 3 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 3 finished in 41.1 seconds.
## Chain 1 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 4 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 1 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 4 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 1 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 4 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 2 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 4 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 2 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 4 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 2 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 4 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 1 finished in 67.1 seconds.
## Chain 4 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%]  (Sampling)
```

```
## Chain 4 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 4 finished in 71.0 seconds.
## Chain 2 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 2 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 2 finished in 271.7 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 112.7 seconds.
## Total execution time: 271.8 seconds.
```

```
dashboard(m2.2)
```

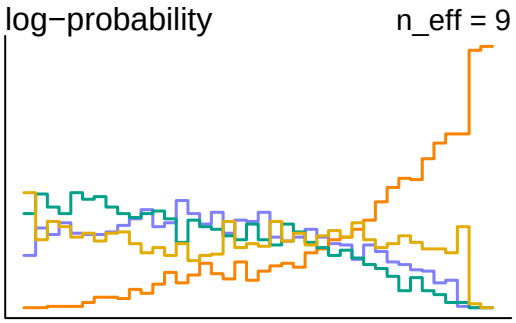




11

Divergent transitions

Check yourself before you wreck yourself



This is still bad model as the sampling efficiency is very small number

```
# Model 2.3
m2.3<- ulam(
    alist (
    # Main model K
        K ~ normal(mu_K, U),
        logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i]  + bK_dK * dK,

        vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
        vector [25]: bK_S <- bK_S_bar + sigma_bK_S * z_bK_S[T],
        vector [25]: bK_SL <- bK_SL_bar + sigma_bK_SL * z_bK_SL[T],
        c(aK_bar, bK_S_bar,  bK_SL_bar) ~ normal(0,0.3),
        c(sigma_aK, sigma_bK_S, sigma_bK_SL) ~ exponential(6),
        z_aK[T] ~ normal(0,0.1),
        z_bK_S[T] ~ normal(0,0.1),
        z_bK_SL[T] ~ normal(0,0.1),
        bK_dK ~ normal(0, 0.3),
        U ~ exponential(1),

    # Model S
        S ~ normal(mu_S, sigma_S),
        logit(mu_S) <- aS[T] + bS_dS * dS,
        vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
        aS_bar ~ normal(0,0.3),
        sigma_aS ~ exponential(6),
        z_aS[T] ~ normal(0,0.1),
        bS_dS ~ normal(0,0.3),
        sigma_S ~ exponential(1),

    # Model SL by Y
```

```
        vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),
    # Model T, This should be hard coded based on the previous research
        T ~ poisson(Y * 25),

    # Model T by Y
        vector[200]:Y ~ normal( 0.5, 0.2)

    ), dat = d, constraints = list(
        SL    = "lower=0,upper=1",
        Y    = "lower=0,upper=1"
    ), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE
)
```

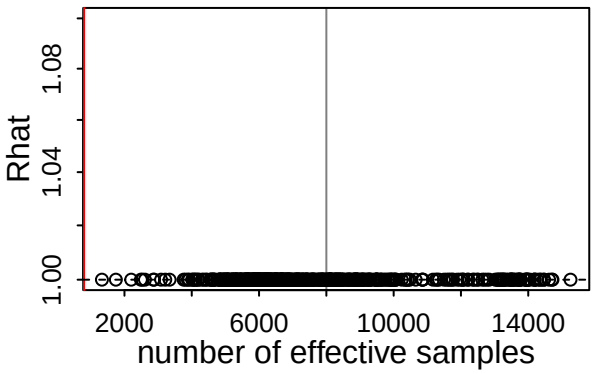
```
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 4 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 1 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 2 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 3 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 4 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 1 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 3 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 4 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 2 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 1 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 4 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 3 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 2 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 1 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 3 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 2 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 4 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 1 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 3 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 4 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 2 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 1 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 3 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 2 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 4 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 1 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 3 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 1 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 2 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 4 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 3 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 1 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 2 Iteration:  800 / 4000 [ 20%]  (Warmup)
```

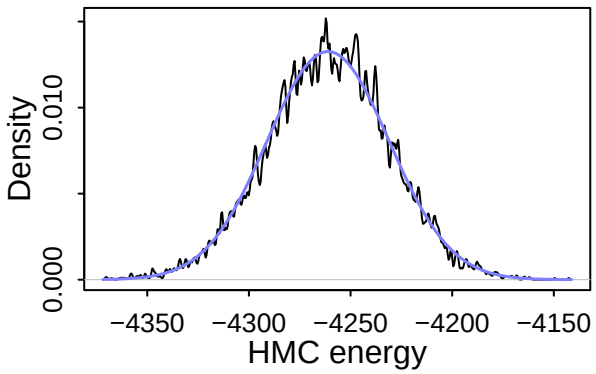
```
## Chain 4 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 3 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 2 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 3 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 4 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 1 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 1 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 2 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 3 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 4 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 3 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 2 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 4 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 2 Iteration: 2001 / 4000 [ 50%]  (Sampling)
```

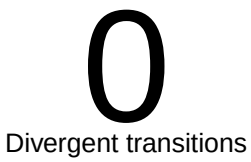
```
## Chain 3 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 4 finished in 29.6 seconds.
```

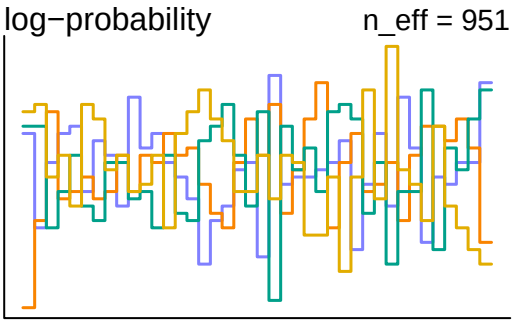
```
## Chain 1 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 1 finished in 35.0 seconds.
## Chain 2 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 3 finished in 36.7 seconds.
## Chain 2 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 2 finished in 37.7 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 34.7 seconds.
## Total execution time: 37.8 seconds.
```

```
dashboard(m2.3)
```










```
precis(m2.3, depth=1)
```

```
##                   mean          sd         5.5%       94.5%      rhat
## bK_SL_bar    1.06880423 0.069825544  0.954891551  1.17778284 1.0012423
## bK_S_bar     0.82221453 0.174900417  0.545237584  1.09956440 1.0002353
## aK_bar      -2.05433545 0.052746441 -2.135420030 -1.96757096 1.0005317
## sigma_bK_SL  0.13863698 0.125063151  0.008593737  0.38289267 1.0012272
## sigma_bK_S   0.16187163 0.155687726  0.009887821  0.46577657 0.9999902
## sigma_aK     0.16661076 0.135705033  0.010643757  0.41987253 1.0008048
## bK_dK       -0.20378199 0.011137071 -0.221395282 -0.18600179 0.9998762
## U            0.01908208 0.002100093  0.015808348  0.02249127 1.0012542
## aS_bar      -0.75877827 0.046994825 -0.833937878 -0.68474250 1.0026145
## sigma_aS     1.72118104 0.255177959  1.361258814  2.15723270 1.0007603
## bS_dS       -0.22259559 0.011478781 -0.241074059 -0.20433737 1.0000607
## sigma_S      0.03306958 0.001764201  0.030361353  0.03598502 1.0008886
##                ess_bulk
## bK_SL_bar    1744.269
## bK_S_bar     4032.951
## aK_bar       3212.485
## sigma_bK_SL  4189.444
## sigma_bK_S   6203.906
## sigma_aK     2488.286
## bK_dK        4835.984
## U            1331.005
## aS_bar       2201.035
## sigma_aS     3920.406
## bS_dS       14619.594
## sigma_S     11766.165
```

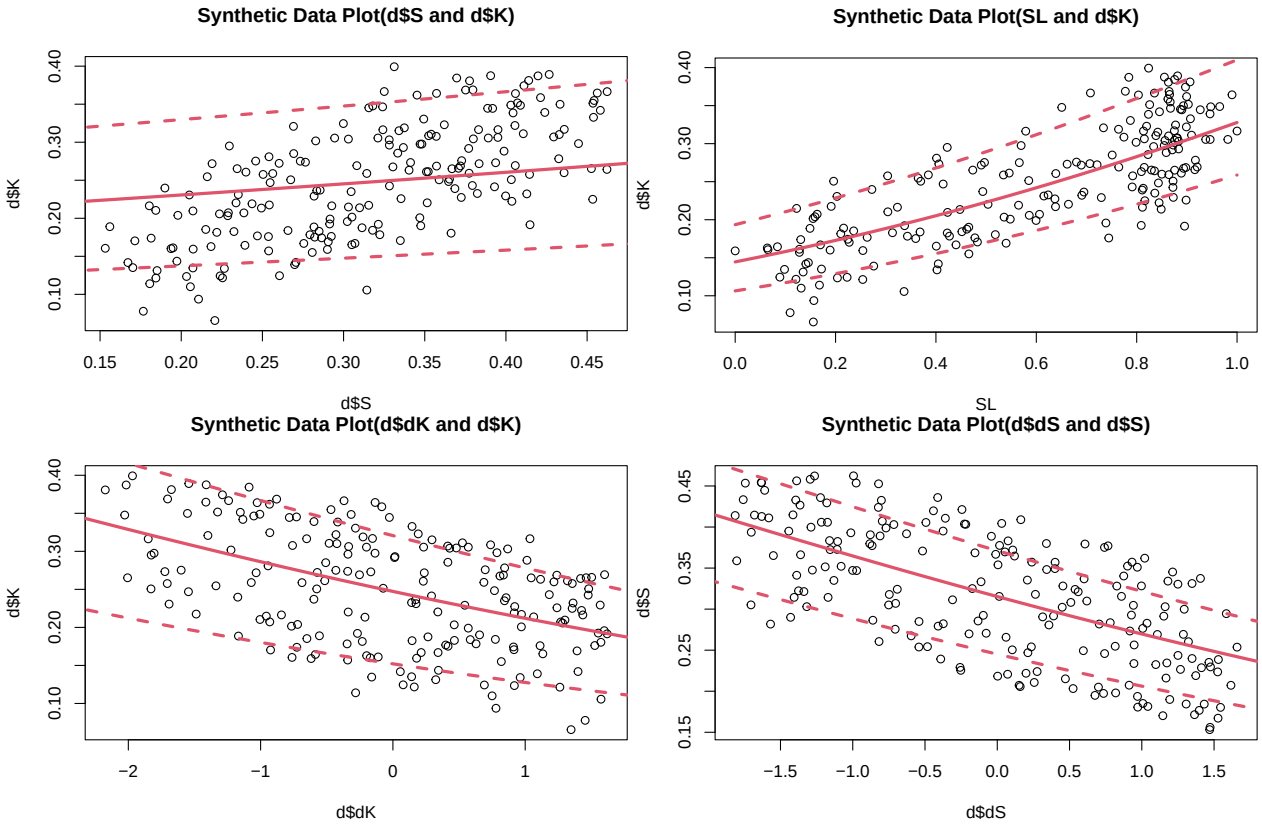
- This is very high eff and no divergent, as I can see in my synthetic data (hopefully, yours is the same„,). This is great. This model is reliably sampled to be used for analysis!

Posterior Check

```
post <- extract.samples(m2.3)
```

- Since I haven't touched hidden parameters since, I do not check them here.

```
plot(d$S, d$K, main = "Synthetic Data Plot(d$S and d$K)")
Sseq<-seq(0, 0.5, len = 20)
KbyS<- sapply(Sseq, # Tricky but t and t to calculate estimates by group, though dK does not have group
    function (i) inv_logit(post$aK + post$bK_S * i + t(t(post$bK_SL) * colMeans(post$SL)) + rep(post$bK
means<- apply( KbyS , 2 , mean )
PIs<- apply( KbyS , 2 , PI )
lines(Sseq, means, col = 2, lwd = 3)
lines(Sseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(Sseq, PIs[2, ], col = 2, lty = 2, lwd = 3)


plot(SL, d$K, main = "Synthetic Data Plot(SL and d$K)")
SLseq<-seq(0, 1, len = 20)
KbySL<- sapply(SLseq,
    function (i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + post$bK_SL * i + rep(post$bK_dK, 25) * d$dK
means<- apply( KbySL , 2 , mean )
PIs<- apply( KbySL , 2 , PI )
lines(SLseq, means, col = 2, lwd = 3)
lines(SLseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(SLseq, PIs[2, ], col = 2, lty = 2, lwd = 3)


plot(d$dK, d$K, main = "Synthetic Data Plot(d$dK and d$K)")
dKseq<-seq(-2.5, 2, len = 20)
KbydK<- sapply(dKseq,
    function (i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + t(t(post$bK_SL) * colMeans(post$SL)) + rep
PIs<- apply( KbydK , 2 , PI )
means<- apply( KbydK , 2 , mean )
lines(dKseq, means, col = 2, lwd = 3)
lines(dKseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dKseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

# Model S

plot(d$dS, d$S, main = "Synthetic Data Plot(d$dS and d$S)")
dSseq<-seq(-2, 2, len = 20)
SbydS<- sapply(dSseq,
    function (i) inv_logit(post$aS + rep(post$bS_dS, 25) * i )) # Used mean of each simulated data
PIs<- apply( SbydS , 2 , PI )
means<- apply( SbydS , 2 , mean )
lines(dSseq, means, col = 2, lwd = 3)
lines(dSseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dSseq, PIs[2, ], col = 2, lty = 2, lwd = 3)
```



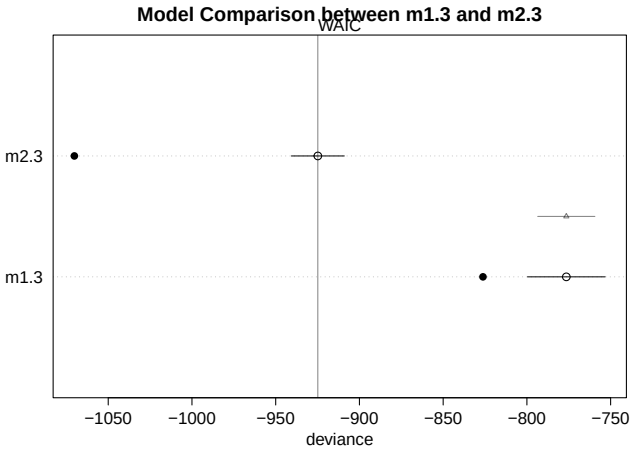
- The model 2.3 looks good. All posterior prediction plots shows inclusion of data points inside the model scope.

Compare Models

```
# Compare models
model_comparison <- compare(m1.3, m2.3)
print(model_comparison)
```

```
##           WAIC       SE    dWAIC      dSE    pWAIC       weight
## m2.3 -924.8423 15.81178   0.0000       NA 72.68939 1.000000e+00
## m1.3 -776.4573 23.19797 148.3849 17.08931 24.88824 6.006514e-33
```

```
# Plot WAIC/PSIS differences
plot(model_comparison, main = "Model Comparison between m1.3 and m2.3")
```



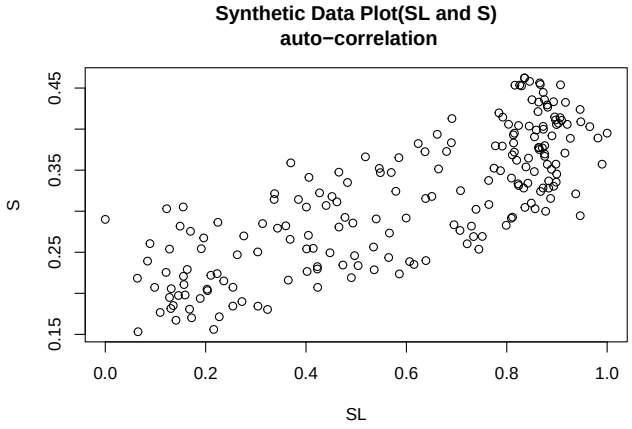
Key Columns in the compare Output:

- WAIC / PSIS: Lower values indicate better out-of-sample predictive accuracy.
- dWAIC / dPSIS: The difference in information criteria between the current model and the top-ranked (best) model.
- dSE: If the standard error is large relative to the difference, the two models have statistically indistinguishable predictive capacities.
- weight: The Akaike weight, which represents the relative probability or evidence that a given model will make the best predictions among the set of compared models

=> Therefore, the model 2.3 is better than the other one with lower WAIC, and so on...

- but also I am wondering if I use correlation between S and SL instead of modeling Y in K model. This is the m2

```
plot(SL, S, main = "Synthetic Data Plot(SL and S)\n auto-correlation")
```



This is because there is an auto-correlation!!

Model 3 (m3)

- see priors in overview.pdf, but this include correlation between SL and S.
- Instead, this does not include multilevel model of Y -> S, because the information is in correlation.
- This code is based on the Ch 14.3 in Statistical Rethinking as SL and S are not centred

```
# Model 3
m3<- ulam(
    alist (
    # Main model K
        K ~ normal(mu_K, U),
```

```
        logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_Y[T] * Y[i] + bK_dK * dK,

        # In this case (m3), we do include correlation between S and SL due to Y.
        vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
        vector [25]: bK_Y <- bK_Y_bar + sigma_bK_Y * z_bK_Y[T],
        c(aK_bar, bK_Y_bar)~normal(0,0.3),
        c(sigma_aK, sigma_bK_Y) ~ exponential(6),
        z_aK[T]~normal(0,0.1),
        z_bK_Y[T] ~ normal(0,0.1),

        transpars>vector[25]:bK_S <<- v[,1],
        transpars>vector[25]:bK_SL <<- v[,2],

        transpars>matrix[25, 2]:v <- compose_noncentered(sigmaSSL, RhoSSL, zSSL),
        vector[2]: sigmaSSL ~ exponential(2),
        cholesky_factor_corr[2]:RhoSSL ~ lkj_corr_cholesky(1), # I do not expect very very strong corre
        matrix[2,25]: zSSL ~ normal(0 , 1),

        # don't need regularization
        bK_dK ~ normal(0, 0.3),
        U ~ exponential(1),

    # Model S
        # This is correlation so I do not include multi-model of Y-> S
        S ~ normal(mu_S, sigma_S),
        logit(mu_S) <- aS[T] + bS_dS * dS ,

        vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
        aS_bar ~ normal(0,0.3),
        sigma_aS ~ exponential(6),
        z_aS[T] ~ normal(0,0.1),

        bS_dS ~ normal(0,0.3),
        sigma_S ~ exponential(1),

    # Model SL by Y
        vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

    # Model T, This should be hard coded based on the previous research
        T ~ poisson(Y * 25),

    # Model T by Y
        vector[200]:Y ~ normal( 0.5, 0.2)
    ), dat = d, constraints = list(
        SL    = "lower=0,upper=1",
        Y     = "lower=0,upper=1"
    ), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(
    adapt_delta = 0.99
  )
)
```

```
## In file included from stan/src/stan/model/model_header.hpp:5:
## stan/src/stan/model/model_base_crtp.hpp:136:8: warning: 'void stan::model::model_base_crtp<M>::write
##   136 |   void write_array(stan::rng_t& rng, Eigen::VectorXd& theta,
```

```
##       |       ^~~~~~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpGKTkkN/model-c3bc701a4564.hpp:1606:3: note:   by 'void ulam_cmd
##  1606 |   write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
##       |   ^~~~~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:205:8: warning: 'void stan::model::model_base_crtp<M>::write_
##   205 |   void write_array(stan::rng_t& rng, std::vector<double>& theta,
##       |        ^~~~~~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpGKTkkN/model-c3bc701a4564.hpp:1606:3: note:   by 'void ulam_cmd
##  1606 |   write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
##       |   ^~~~~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:154:17: warning: 'double stan::model::model_base_crtp<M>::log
##   154 |   inline double log_prob(std::vector<double>& theta, std::vector<int>& theta_i,
##       |                 ^~~~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpGKTkkN/model-c3bc701a4564.hpp:1645:3: note:
## by 'T_ ulam_cmdstanr_a684a685757990b7a510e9ec4271b65b_model_namespace::ulam_cmdstanr_a684a685757990b7
##  1645 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:91:17: warning: 'double stan::model::model_base_crtp<M>::log_
##    91 |   inline double log_prob(Eigen::VectorXd& theta,
##       |                 ^~~~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpGKTkkN/model-c3bc701a4564.hpp:1645:3: note:   by 'T_ ulam_cmdst
##  1645 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:159:20: warning: 'stan::math::var stan::model::model_base_crt
##   159 |   inline math::var log_prob(std::vector<math::var>& theta,
##       |                    ^~~~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpGKTkkN/model-c3bc701a4564.hpp:1645:3: note:   by 'T_ ulam_cmdst
## trix<T_a, -1, 1>&, std::ostream*) const'
##  1645 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~~
## stan/src/stan/model/model_base_crtp.hpp:96:20: warning: 'stan::math::var stan::model::model_base_crtp
##    96 |   inline math::var log_prob(Eigen::Matrix<math::var, -1, 1>& theta,
##       |                    ^~~~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpGKTkkN/model-c3bc701a4564.hpp:1645:3: note:   by 'T_ ulam_cmdst
##  1645 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##       |   ^~~~~~~~
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppDa
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 1
## Chain 2 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 2 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppDa
```

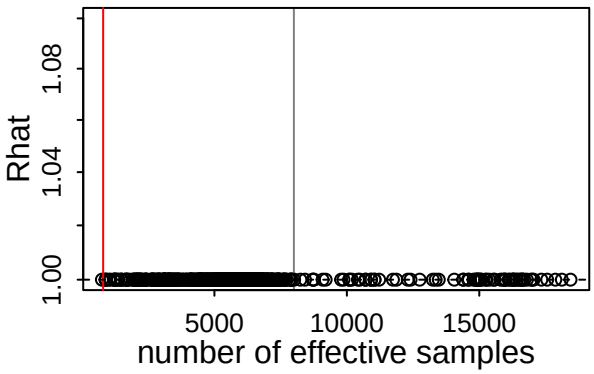
```
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 2
## Chain 3 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppDa
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 3
## Chain 4 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppDa
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or mi
## Chain 4
## Chain 1 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 4 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 3 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 2 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 2 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 4 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 3 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 1 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 2 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 4 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 2 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 1 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 3 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 4 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 1 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 3 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 2 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 4 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 1 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 2 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 4 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 1 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 4 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 2 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 1 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 3 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 4 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 1 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 2 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 3 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 1 Iteration:  900 / 4000 [ 22%]  (Warmup)
```

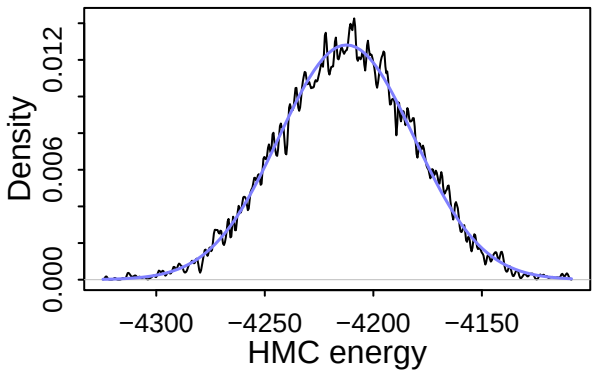
```
## Chain 3 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 2 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 4 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 3 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 3 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 4 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 3 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 1 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 2 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 3 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 1 Iteration: 2200 / 4000 [ 55%]  (Sampling)
```

```
## Chain 4 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 3 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%]  (Sampling)
```

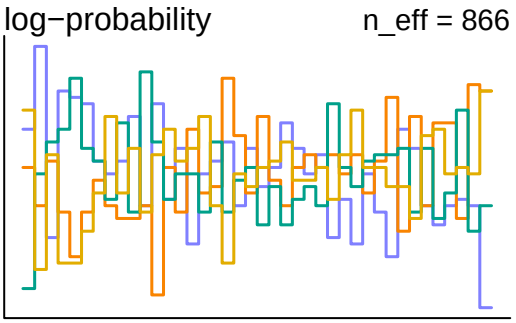
```
## Chain 1 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 1 finished in 120.5 seconds.
## Chain 3 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 3 finished in 126.3 seconds.
## Chain 4 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 4 finished in 127.7 seconds.
## Chain 2 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 2 finished in 160.1 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 133.7 seconds.
## Total execution time: 160.2 seconds.
```

```
dashboard(m3)
```





Outlook good



-> Looks bad sampling efficiency (as I can see here, but might be different for you due to randomized data).

Therefore, I will tackle this problem with the same approach though I do not want to remove the correlation parameters. Let me remove one more thing so that it corresponds m1.3.

```
# Model 3.2
m3.2<- ulam(
    alist (
    # Main model K
        K ~ normal(mu_K, U),
        logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_dK * dK,
        vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
        aK_bar~normal(0,0.3),
        sigma_aK ~ exponential(6),
        z_aK[T]~normal(0,0.1),
        transpars>vector[25]:bK_S <<- v[,1],
        transpars>vector[25]:bK_SL <<- v[,2],
        transpars>matrix[25, 2]:v <- compose_noncentered(sigmaSSL, RhoSSL, zSSL),
        vector[2]: sigmaSSL ~ exponential(2),
        cholesky_factor_corr[2]:RhoSSL ~ lkj_corr_cholesky(1),
        matrix[2,25]: zSSL ~ normal(0 , 1),
        bK_dK ~ normal(0, 0.3),
        U ~ exponential(1),

    # Model S
        S ~ normal(mu_S, sigma_S),
        logit(mu_S) <- aS[T] + bS_dS * dS ,
        vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
        aS_bar ~ normal(0,0.3),
        sigma_aS ~ exponential(6),
        z_aS[T] ~ normal(0,0.1),
        bS_dS ~ normal(0,0.1),
```

```
        sigma_S ~ exponential(1),

    # Model SL by Y
        vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

    # Model T
        T ~ poisson(Y * 25),

    # Model T by Y
        vector[200]:Y ~ normal( 0.5, 0.2)
    ), dat = d, constraints = list(
        SL    = "lower=0,upper=1",
        Y    = "lower=0,upper=1"
    ), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(
    adapt_delta = 0.99
  )
)
```

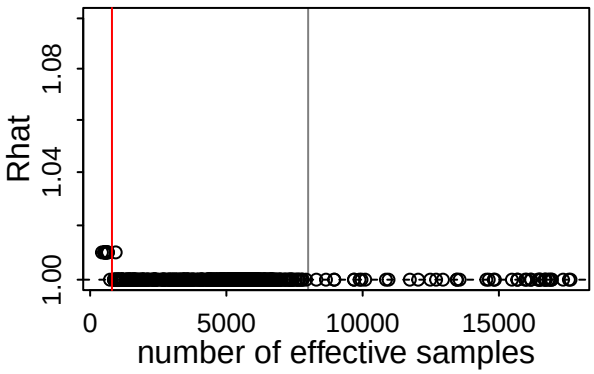
```
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 4000 [  0%]  (Warmup)
## Chain 1 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 2 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 3 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 4 Iteration:  100 / 4000 [  2%]  (Warmup)
## Chain 1 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 4 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 3 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 4 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 3 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 2 Iteration:  200 / 4000 [  5%]  (Warmup)
## Chain 4 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 1 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 3 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 3 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 1 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 4 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 3 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 2 Iteration:  300 / 4000 [  7%]  (Warmup)
## Chain 4 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 1 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 3 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 2 Iteration:  400 / 4000 [ 10%]  (Warmup)
## Chain 4 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 1 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 3 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 4 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 1 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 3 Iteration:  900 / 4000 [ 22%]  (Warmup)
```

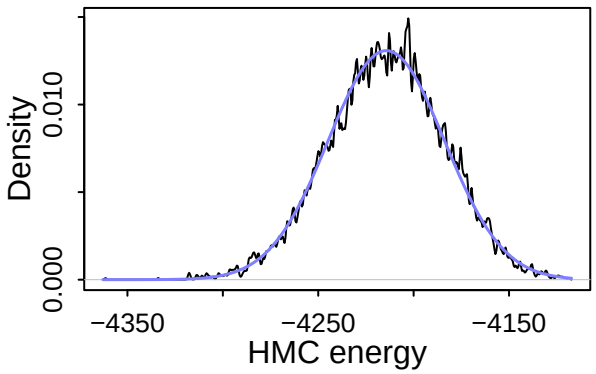
```
## Chain 1 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 2 Iteration:  500 / 4000 [ 12%]  (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 4 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 2 Iteration:  600 / 4000 [ 15%]  (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 1 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 2 Iteration:  700 / 4000 [ 17%]  (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 2 Iteration:  800 / 4000 [ 20%]  (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 2 Iteration:  900 / 4000 [ 22%]  (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%]  (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%]  (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%]  (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 3 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%]  (Warmup)
## Chain 3 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 1 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%]  (Warmup)
## Chain 3 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 4 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 2 Iteration: 1500 / 4000 [ 37%]  (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 3 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 2 Iteration: 1600 / 4000 [ 40%]  (Warmup)
## Chain 4 Iteration: 2100 / 4000 [ 52%]  (Sampling)
```

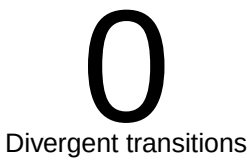
```
## Chain 3 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 2 Iteration: 1700 / 4000 [ 42%]  (Warmup)
## Chain 4 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 2 Iteration: 1800 / 4000 [ 45%]  (Warmup)
## Chain 3 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 2 Iteration: 1900 / 4000 [ 47%]  (Warmup)
## Chain 3 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 2 Iteration: 2000 / 4000 [ 50%]  (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%]  (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%]  (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%]  (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%]  (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%]  (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%]  (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 3 finished in 86.5 seconds.
## Chain 1 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%]  (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%]  (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%]  (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%]  (Sampling)
```

```
## Chain 2 Iteration: 2900 / 4000 [ 72%]  (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%]  (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%]  (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%]  (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%]  (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%]  (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%]  (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%]  (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 4 finished in 124.8 seconds.
## Chain 1 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%]  (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%]  (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 1 finished in 128.4 seconds.
## Chain 2 Iteration: 3900 / 4000 [ 97%]  (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%]  (Sampling)
## Chain 2 finished in 133.9 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 118.4 seconds.
## Total execution time: 134.1 seconds.
```

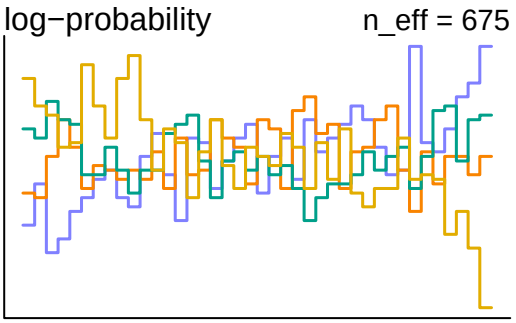
```
dashboard(m3.2)
```







Outlook good




```
precis(m3.2, depth=1)
```

```
##                 mean          sd        5.5%       94.5%     rhat  ess_bulk
## aK_bar    -1.35539867 0.071411423 -1.46895208 -1.24279637 1.000901  2195.982
## sigma_aK   0.28242407 0.325119107  0.01281582  0.94392658 1.001790  3041.913
## bK_dK     -0.20502788 0.011713421 -0.22386970 -0.18587077 1.001087 10854.235
## U          0.02676545 0.002776712  0.02221697  0.03098383 1.008419   546.931
## aS_bar    -0.75934133 0.047045855 -0.83413142 -0.68358483 1.000626  2022.462
## sigma_aS   1.71364687 0.252937966  1.34566052  2.14951585 1.001519  3120.322
## bS_dS     -0.21988323 0.011353104 -0.23790199 -0.20181886 1.000265 14540.211
## sigma_S    0.03303787 0.001706306  0.03044304  0.03592634 0.999890 13462.260
```

- This model is good enough.

Posterior Check

```
post <- extract.samples(m3.2)
```

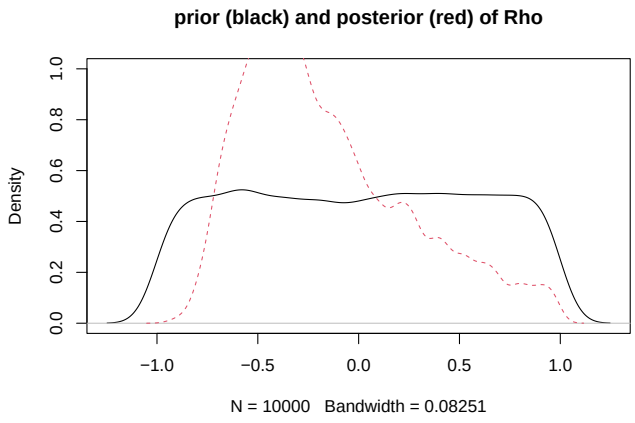
- I do not check the Hidden parameters since I have never touched it since m1

Correlation?

- Rho value shows a bit of correlation between S and SL.

```
R <- rlkjcorr( 1e4 , K=2 , eta=1 )# eta min lkj_corr()
plot(density( R[,1,2]), ylim = c(0, 1), main = "prior (black) and posterior (red) of Rho")
# This is complicated as cholesky correlation
true_rho_samples=NULL
for (i in 1:2000) {
  L <- post$RhoSSL[i, , ]
  R <- L %*% t(L)
  true_rho_samples[i] <- R[1, 2]
```

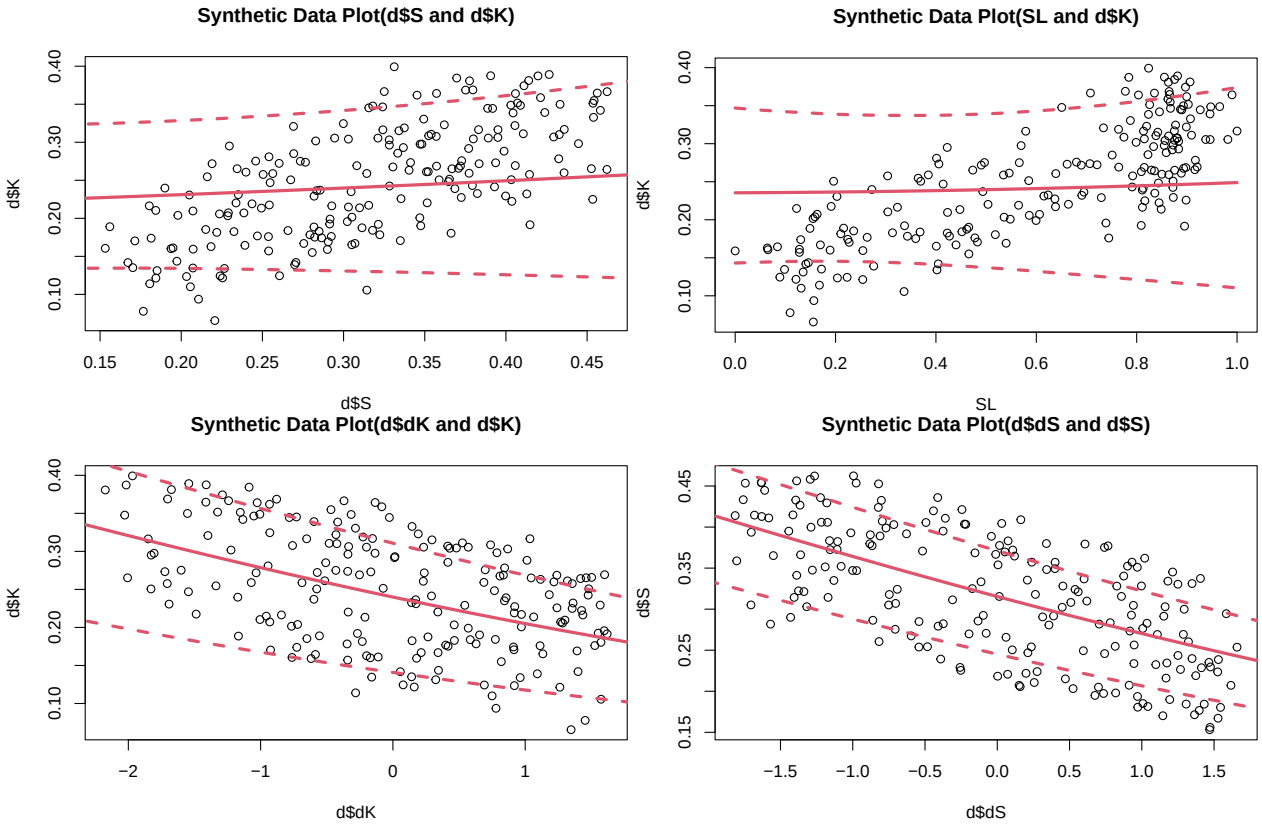
```
}
dens(true_rho_samples, add = TRUE, lty = 2, col = 2)
```



Posterior Predictions

```
plot(d$S, d$K, main = "Synthetic Data Plot(d$S and d$K)")
Sseq<-seq(0, 0.5, len = 20)
KbyS<- sapply(Sseq, # Tricky but t and t to calculate estimates by group, though dK does not have group
    function (i) inv_logit(post$aK + post$bK_S * i + t(t(post$bK_SL) * colMeans(post$SL)) + rep(post$bK_
means<- apply( KbyS , 2 , mean )
PIs<- apply( KbyS , 2 , PI )
lines(Sseq, means, col = 2, lwd = 3)
lines(Sseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(Sseq, PIs[2, ], col = 2, lty = 2, lwd = 3)


plot(SL, d$K, main = "Synthetic Data Plot(SL and d$K)")
SLseq<-seq(0, 1, len = 20)
KbySL<- sapply(SLseq,
    function (i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + post$bK_SL * i + rep(post$bK_dK, 25) * d$dI
means<- apply( KbySL , 2 , mean )
PIs<- apply( KbySL , 2 , PI )
lines(SLseq, means, col = 2, lwd = 3)
lines(SLseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(SLseq, PIs[2, ], col = 2, lty = 2, lwd = 3)


plot(d$dK, d$K, main = "Synthetic Data Plot(d$dK and d$K)")
dKseq<-seq(-2.5, 2, len = 20)
KbydK<- sapply(dKseq,
    function (i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + t(t(post$bK_SL) * colMeans(post$SL)) + rep
PIs<- apply( KbydK , 2 , PI )
means<- apply( KbydK , 2 , mean )
lines(dKseq, means, col = 2, lwd = 3)
lines(dKseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dKseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

# Model S
plot(d$dS, d$S, main = "Synthetic Data Plot(d$dS and d$S)")
dSseq<-seq(-2, 2, len = 20)
SbydS<- sapply(dSseq,
    function (i) inv_logit(post$aS + rep(post$bS_dS, 25) * i )) # Used mean of each simulated data
```

```
PIs<- apply( SbydS , 2 , PI )
means<- apply( SbydS , 2 , mean )
lines(dSseq, means, col = 2, lwd = 3)
lines(dSseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dSseq, PIs[2, ], col = 2, lty = 2, lwd = 3)
```



Overall K estimation is fine.

let me compare all those models.

Compare Models

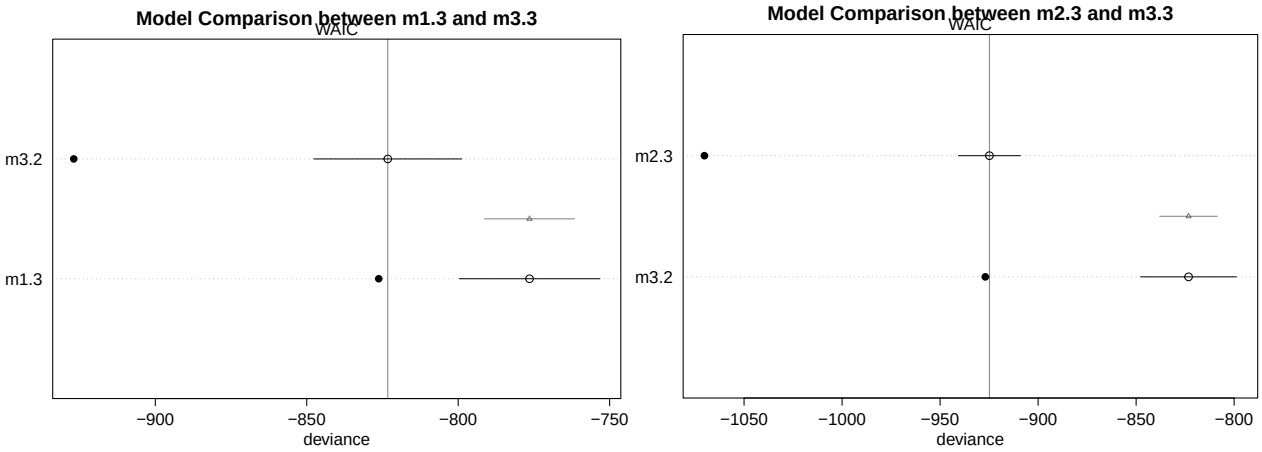
```
# Compare models
model_comparison <- compare(m1.3, m3.2)
print(model_comparison)
```

```
##           WAIC       SE    dWAIC      dSE    pWAIC       weight
## m3.2 -823.2516 24.41720  0.00000       NA 51.83992 1.000000e+00
## m1.3 -776.4573 23.19797 46.79426 14.90919 24.88824 6.898523e-11
```

```
# Plot WAIC/PSIS differences
plot(model_comparison, main = "Model Comparison between m1.3 and m3.3")

model_comparison <- compare(m2.3, m3.2)
print(model_comparison)
```

```
##           WAIC       SE    dWAIC      dSE    pWAIC       weight
## m2.3 -924.8423 15.81178   0.0000       NA 72.68939 1.000000e+00
## m3.2 -823.2516 24.41720 101.5907 14.68786 51.83992 8.706957e-23
```

```r
# Plot WAIC/PSIS differences
plot(model_comparison, main = "Model Comparison between m2.3 and m3.3")
```



This looks Model 2.3 is a bit better in this case.

Future Suggestions

- Removing causal relationship in each model (for example, no causal relationship from Y to K, . . .) may be examined with actual data.
- Model 1 can be used without removing any parameters, since the model somtimes works fine.

```r
rm(list=ls())
```